FAQ-03: OneAgent and OpenTelemetry — Choosing Per Workload

> **Series:** FAQ — Frequently Asked Questions | **Reference:** 03 — OneAgent and OpenTelemetry — Choosing Per Workload | **Created:** May 2026 | **Last Updated:** 10/15/2026

Overview

The recurring question:

Three related questions show up repeatedly. Teams already running OpenTelemetry ask: *Is it worth the effort to convert our existing OTel-instrumented services to Dynatrace OneAgent?* And the sharper version: *If our OTel implementation is working today, do we need OneAgent at all?* And from teams starting fresh with nothing instrumented yet: *We have a clean slate — which should we install first, OneAgent or OpenTelemetry?*

The framing that answers all three: OpenTelemetry is the open, vendor-neutral foundation for telemetry — common semantic conventions, reusable integrations, instrumentation that travels across environments and tools. OneAgent adds automation and runtime depth on top of that foundation: instrumentation with no code change, a centrally managed lifecycle, topology, and capabilities that need runtime access — Application Security, Live Debugger and automatic RUM injection. Dynatrace documents the two as working together, not as alternatives, so the useful question is never *OneAgent or OpenTelemetry* but *what fits each workload*: OTel only, OneAgent only, or both. Each is a supported end state, and mixed estates are normal.

All three come down to a level-of-effort versus capability-gain trade-off, and all three deserve a frank answer. The answer is the same across every runtime Dynatrace supports — Java, Kotlin, Scala, Groovy, .NET, Node.js, Python, Go, PHP, Ruby — though a handful of runtimes have edge cases (covered in §7 and §14).

Short answer (effort-vs-conversion): If an application is already emitting OpenTelemetry data and runs somewhere OneAgent can reach (long-lived hosts and containers, AWS Lambda through the Dynatrace Lambda layer, and the supported Azure Functions plans — §4), **the most cost-effective path is almost always to keep OTel for application instrumentation and add OneAgent alongside it for host, runtime, and infrastructure coverage** — not to convert. The two are complementary, not competing. A wholesale conversion is usually a *deletion* exercise, not a re-instrumentation exercise — and even then, the gain is rarely worth the disruption unless a specific Dynatrace capability is needed that OTel cannot provide on its own.

Short answer (is OneAgent required at all): No, not strictly. OTel alone can fully instrument an application and ship to Dynatrace via OTLP — Davis AI works on that data, traces correlate via W3C `traceparent`, and metrics/logs land in Grail. OTel-only is a complete path: Dynatrace detects services and service topology from OTel spans, and the OpenTelemetry Host Monitoring extension turns Collector host metrics into host and process entities. What OneAgent adds is fully managed collection — runtime internals (heap / GC / threads / event loop), code-level method tracing, and infrastructure topology without operating a Collector — plus capabilities that need runtime access: Application Security, Live Debugger and automatic RUM injection. Whether those matter depends on the workload — see §2 for the side-by-side capability table.

Short answer (greenfield — which one first?): It depends on what you have already decided. **If your organization has chosen OpenTelemetry**, instrument with OTel — Dynatrace ingests it natively and detects services, endpoints and service topology from it, and nothing in this FAQ argues you off that choice. **If you have no preference** and the workload runs on a standard OneAgent-supported runtime (Java / .NET / Node.js / Python / PHP / Go) on long-lived hosts or containers, **OneAgent is the lowest-effort start**: fully managed, zero code change, time-to-first-trace in minutes. Add OTel when you need custom business spans or open-standard telemetry; the two coexist by design (see §8). Some workloads are OTel by necessity whichever you prefer: **serverless platforms OneAgent does not reach** (GCP Cloud Functions, and Azure Functions plans outside the supported list — §4), **Ruby** application code (OneAgent does not inject it), and **async-fragmenting runtimes** like Scala effect systems (otel4s / zio-telemetry are technically required — see §7).

This FAQ frames the trade-offs, the hidden costs, and the workload-specific edge cases — including **async-fragmenting runtimes** (effect systems, coroutines, async/await without explicit context propagation) where the choice has a clear technical answer rather than a preference.

> **Sources:** Extend Dynatrace with OpenTelemetry (DT docs) — *"adopting the open standards of OpenTelemetry where openness matters most"*; *"regardless of how you mix your instrumentation choices"*, Dynatrace OpenTelemetry Collector (Dynatrace blog, 03/18/2024) — *"Used together, Dynatrace extends OpenTelemetry observability, and OpenTelemetry extends Dynatrace observability."*

Table of Contents

1. Framing the Question Correctly
2. Is OneAgent Required at All?
3. What Each Tool Actually Is
4. Capability Comparison
5. What "Convert to OneAgent" Really Means
6. Framework Auto-Instrumentation Coverage Across Runtimes
7. Async Context Propagation Across Runtimes
8. The Coexistence Pattern (How They Work Together in Dynatrace)
9. Decision Tree by Workload Type
10. Three Migration Paths and Their Effort
11. Pros and Cons Side-by-Side
12. Is the Conversion Worth the Effort?
13. Recommended Approach
14. Deploying to Windows Servers
15. Runtime Applicability Map

Prerequisites

Requirement	Details
Audience	Engineering leads, SRE/observability architects, platform owners evaluating instrumentation strategy
Format	Decision-support FAQ — no DQL or hands-on labs
Assumed knowledge	Familiarity with the basics of distributed tracing, the difference between an *agent* and an *SDK*, and what your services run on (long-lived host/container vs. serverless)
Scope	All major runtimes Dynatrace supports — Java, Kotlin, Scala, Groovy, .NET (C# / F# / VB), Node.js (JS / TS), Python, Go, PHP, Ruby. Runtime-specific edge cases (effect systems, coroutines, async/await context propagation) are called out in §7

1. Framing the Question Correctly

Customers usually ask this as **OneAgent OR OpenTelemetry** — a binary choice. That framing is the first mistake. The two tools answer different questions:

- **OneAgent** answers: *"What is happening on my hosts, in my application runtime (JVM / CLR / V8 / CPython / Go runtime / Zend / Ruby VM), and in the frameworks I'm running — without me writing any instrumentation code?"*
- **OpenTelemetry** answers: *"How do I emit portable, vendor-neutral, code-level telemetry from my application — including the parts no agent can see automatically?"*

Re-framed correctly, the question becomes: **"Given we already have OTel, do we still need OneAgent? And if we add OneAgent, do we need to remove OTel?"** The answer to the first is *usually yes* (for non-serverless workloads). The answer to the second is *almost never*.

Dynatrace frames the platform side the same way: OTel data and OneAgent data are analyzed together *"regardless of how you mix your instrumentation choices"* — OneAgent on some hosts, OTel on others, both on many. A mixed estate is the expected end state, not a transition to be finished.

> **Sources:** Use OneAgent with OpenTelemetry (DT docs), Extend Dynatrace with OpenTelemetry (DT docs) — *"regardless of how you mix your instrumentation choices"*.

2. Is OneAgent Required at All?

Short answer: No. OTel-only is a complete, supported path. Add OneAgent where its fully managed collection or runtime-access capabilities are worth it to you.

OpenTelemetry alone can fully instrument an application and ship to Dynatrace via OTLP. Traces correlate via W3C `traceparent`, metrics and logs land in Grail, and Davis AI operates on that data. For serverless platforms OneAgent does not reach (GCP Cloud Functions, unsupported Azure Functions plans — §4) and async-fragmenting runtimes that lack agent-side context propagation, **OTel is the application-instrumentation layer** — on those platforms OneAgent cannot run, and on those runtimes its tracing fragments (§7). A multi-backend or open-standards mandate is a different kind of answer: it settles *who owns application instrumentation* (OTel), not whether OneAgent adds value alongside it on the hosts and containers it reaches.

What each path gives you in Dynatrace:

Capability	OTel-only	With OneAgent added
Application traces / metrics / logs	Yes	—
Runtime internals (heap, GC, threads, event loop, allocations)	Partial — needs runtime-telemetry receiver wired per service (JMX for JVM, EventCounters for .NET, perf_hooks/prom-client for Node, psutil for Python, runtime/metrics for Go)	Yes — automatic
Host metrics (CPU, memory, disk, network)	Partial — needs OTel Collector with hostmetrics receiver deployed on every node	Yes — automatic
Process / container correlation	Manual attribute hygiene	Yes — automatic
Smartscape topology and dependency map	Services and service-to-service calls detected from OTel spans; host and process entities via the OpenTelemetry Host Monitoring extension	Automatic for every monitored host, process, container and service
PurePath code-level method tracing on demand	Not available	Yes
Real-User Monitoring ↔ backend trace linking	XHR/Fetch and mobile requests link through W3C trace context; HTML-document and resource requests cannot link	All request types, including HTML-document and resource requests (Server-Timing header)
Auto-instrumentation of frameworks OTel doesn't cover (proprietary middleware, exotic app servers)	Gap	Yes — often covered
Davis AI	Runs on OTel data in Grail, including service topology; results depend on clean semantic-convention attributes	Same Davis AI, with infrastructure topology filled in automatically
Application Security (Runtime Vulnerability Analytics, Runtime Application Protection)	Not from OTel telemetry — both run on OneAgent deep monitoring	Yes
Live Debugger	Not from OTel telemetry	Yes — Java and Node.js, OneAgent 1.309+
RUM JavaScript injection	Manual or agentless insertion	Automatic in full-stack mode
Update cadence	Per-application redeploy	Centrally managed; no app redeploy

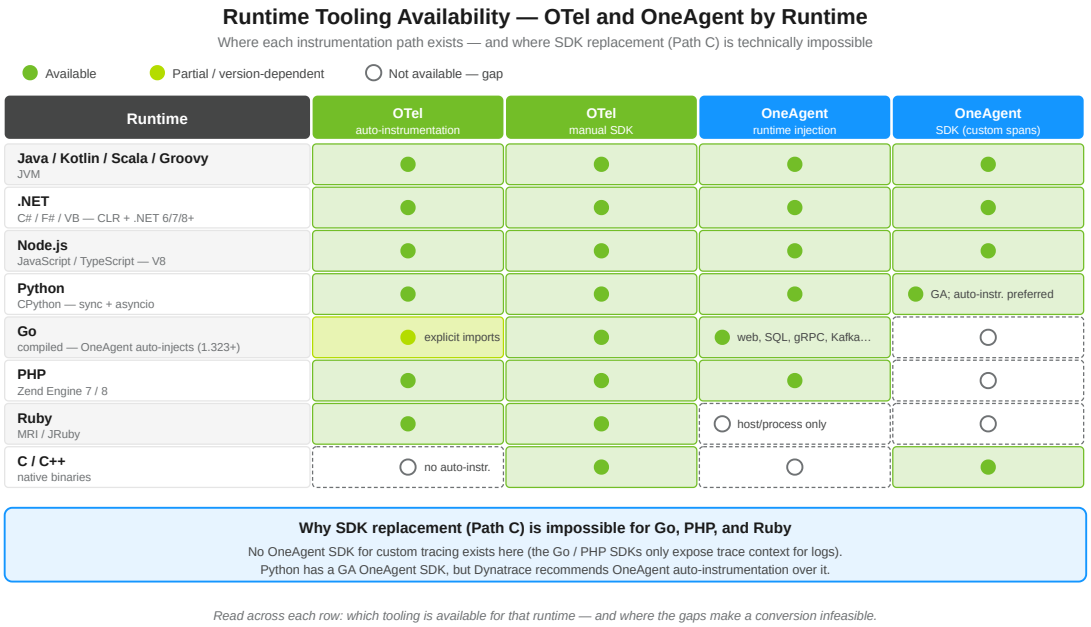
Decision rule in three lines:

- **Serverless platform OneAgent does not reach (§4), or async-fragmenting runtime without OTel hooks** → OTel for application telemetry. OneAgent can't run there, or its tracing fragments — it still adds host and runtime signals wherever it does run. A **multi-backend or open-standards mandate** puts OTel in charge of application instrumentation and leaves OneAgent free to add depth alongside it.
- **Standard workloads on long-lived hosts/containers, and fully managed collection, Application Security or Live Debugger matter** → add OneAgent alongside OTel. The install is a deployment task — no code change — and the two coexist by design (see §8 for the Span Sensor vs OTLP patterns).
- **OTel is your collection strategy, or it is working today** → staying OTel-only is a complete, supported choice. The application telemetry is all there; what you forgo is OneAgent's automation and runtime-access capabilities. Revisit workload by workload when one of them is needed.

Honest framing: OneAgent is not *required* — OTel data is fully usable in Dynatrace on its own. What OneAgent adds comes in two parts. The first is fully managed collection: host, runtime and infrastructure-topology coverage without operating a Collector fleet, receivers and an attribute strategy yourself. In community practice that is often a total-cost-of-ownership decision more than a feature decision. The second is runtime depth: PurePath, Application Security, Live Debugger and automatic RUM injection, which need runtime access and have no OTel equivalent today. If your Collector estate already runs well, the first part shrinks; the second does not.

> **Sources:**
>
> - Use OneAgent with OpenTelemetry (DT docs)
> - Frontend-backend linking (DT docs) — *"It works for distributed traces captured by OneAgent as well as those captured by OpenTelemetry."*, *"For web frontends with an OTel-instrumented backend, only XHR and Fetch requests can be linked to backend traces."* and *"Requests for the HTML document and its included resources can't be linked, because setting the Server-Timing header is not part of the OpenTelemetry specification."*
> - Ingest OpenTelemetry — getting started (DT docs)
> - Technology support (DT docs)
> - Unified services (DT docs) — services detected with Service Detection v2 *"were first introduced for OpenTelemetry"*; *"Automatic endpoint detection and monitoring"*
> - OpenTelemetry Host Monitoring extension (DT docs) — *"This extension automatically generates entities for hosts and their running processes."*
> - Verified on a live tenant 10/13/2026: six services sending only OTLP each had a Smartscape `SERVICE` node, with 9 service-to-service `calls` edges between them and no host, process or Kubernetes edges (`smartscapeNodes "SERVICE" | traverse`).
> - Runtime Vulnerability Analytics (DT docs) — *"Dynatrace OneAgent monitors loaded libraries, runtime components, and data flows in real time."*
> - Runtime Application Protection (DT docs) — *"Code-level visibility provided by OneAgent"*
> - Live Debugger (DT docs) — *"OneAgent version 1.309+ with Java Live-Debugger and Node.js Live-Debugger"*
> - RUM JavaScript injection (DT docs) — *"Once OneAgent is deployed in full-stack monitoring mode on a host, it by default automatically injects the RUM JavaScript"*
> - OneAgent SDK for Java (Dynatrace GitHub) — *"not supported on serverless code modules .. Consider using OpenTelemetry instead"* (a statement about the **SDK**, not about OneAgent itself — see §4)

3. What Each Tool Actually Is



Read across each row: which tooling is available for that runtime — and where the gaps make a conversion infeasible.

OneAgent is a Dynatrace-installed binary that runs on the host (or as a container sidecar/initcontainer in Kubernetes via the Dynatrace Operator). When the application runtime starts, OneAgent injects itself and auto-instruments a long list of frameworks and runtimes. It produces *PurePath* traces, host metrics, runtime metrics, log streams, process snapshots, and Smartscape topology — all without code changes. OneAgent's runtime injection covers Java/Kotlin/Scala/Groovy (JVM), .NET (CLR), Node.js (V8), PHP (Zend), Python, and Go (64-bit Go executables on x86 from OneAgent 1.323, and on AArch64 — `net/http`, SQL drivers, gRPC, Kafka, MongoDB, Redis, Cassandra and AWS SDK clients per the Go section of the technology-support matrix; on Linux the binary must be dynamically linked unless you use Go static monitoring); for Ruby, which it does not inject, OneAgent still captures host/process telemetry around it.

OpenTelemetry (OTel) is a CNCF specification with a per-runtime delivery model. The two main consumption modes vary slightly per runtime:

Runtime	OTel auto-instrumentation entry point	OTel manual SDK entry point
Java / Kotlin / Scala / Groovy (JVM)	`-javaagent:opentelemetry-javaagent.jar`	`io.opentelemetry.api` (otel4s, zio-telemetry, kotlinx-coroutines wrappers)
.NET	`OpenTelemetry.Instrumentation.*` NuGet packages auto-wired in `Program.cs`, or the .NET auto-instrumentation injector	`System.Diagnostics.ActivitySource` / OTel SDK
Node.js (JS / TS)	`--require @opentelemetry/auto-instrumentations-node/register`	`@opentelemetry/api`
Python	`opentelemetry-instrument <command>` (CLI wrapper) or programmatic auto-instrumentation	`opentelemetry-api` / `opentelemetry-sdk`
Go	No bytecode injection — explicit `otelhttp`, `otelgin`, `otelgrpc`, etc. middleware imports	`go.opentelemetry.io/otel`
PHP	`opentelemetry` PECL extension + auto-instrumentation Composer packages	`open-telemetry/api` Composer package
Ruby	`opentelemetry-instrumentation-all` gem	`opentelemetry-api` / `opentelemetry-sdk`

These OTel modes are independent — you can use any combination, or none. "Already instrumented with OTel" usually means a mix: the auto-instrumentation for HTTP/DB plus some manual SDK calls for business logic.

JVM has a third delivery model. The auto-instrumentation column above shows the static `-javaagent:` flag, but for JVM workloads there are actually three distinct OTel delivery options, with different operational profiles:

Mode	How it loads	Operational characteristics
1. **SDK in code**	OTel dependencies in `pom.xml` / `build.gradle`; explicit `Tracer` calls in source	Tied to the application build; redeploy needed for instrumentation changes; full control over what is captured
2. **Static `-javaagent`**	JVM startup flag points at `opentelemetry-javaagent.jar` — auto-instrumentation kicks in before `main()`	Coupled to the JVM start command; if the agent JAR fails to load (bad version, classpath conflict, missing file), the JVM may fail to start, taking the line-of-business application down with it
3. **Runtime attach**	`opentelemetry-runtime-attach` (OTel Java contrib) — the application calls `RuntimeAttach.attachJavaagentToCurrentJvm()` at the beginning of its `main()` method, and the embedded agent attaches to the running JVM	For when the JVM arguments cannot be changed. Needs a code dependency and a full JDK (not a JRE); the call must come from `main()` on the main thread

Choosing between them. The OTel project positions runtime attach as the route for teams that cannot update the JVM arguments, not as a production default; the OTel pages cited here do not rank the three modes. In community practice, the concern with option 2 is that it puts the agent on the JVM's startup path (a bad JAR or classpath conflict can stop the JVM starting), while option 3 trades that for a code dependency and a JDK requirement — weigh both against your own deployment constraints.

OneAgent takes a different route from all three: its Java code module is loaded as a native JVMTI agent **at process start, through automatic process injection** — no `-javaagent` flag in your start command, but also no attach to a JVM that is already running. That is why a new code-module version, or a change to OneAgent's connection settings, takes effect only when the monitored process restarts. It is also why OneAgent updates do not require application redeploys: the start command never references the agent, so a new agent version is picked up at the next workload restart without touching it.

Dynatrace OneAgent SDKs (separate, narrow libraries — *not* OneAgent itself) exist per language for adding custom traces inside an application that is *already running under OneAgent*. With no OneAgent attached, the SDK calls become no-ops.

Runtime	OneAgent SDK	Status
Java	OneAgent-SDK-for-Java	Active — version 1.9.0; explicitly recommends OpenTelemetry for serverless
.NET	OneAgent-SDK-for-DotNet	GA — .NET Standard 1.0 (Full Framework 4.5+ / .NET Core 1.0+); v1.8.0 last release
Node.js	OneAgent-SDK-for-NodeJs	Active
C / C++	OneAgent-SDK-for-C	Active
Python	OneAgent-SDK-for-Python	GA — v1.5.2 (01/2026): remote calls, SQL, incoming/outgoing web requests, messaging, custom services, custom request attributes. Dynatrace's Python page recommends migrating from it to OneAgent auto-instrumentation
Go	OneAgent-SDK-for-Go	GA — log and trace **enrichment metadata only**, no custom tracing. OneAgent auto-injects Go executables and the Span Sensor captures OTel Go spans, so custom spans come from OTel
PHP	OneAgent-SDK-for-PHP	Early adopter — returns the current Trace ID and Span ID, for log enrichment; no custom tracing. Custom spans come from OTel
Ruby	None supported (the only Ruby binding is an archived, unsupported sample; Dynatrace recommends the Ruby OpenTelemetry stack for code-level traces)	—

Implication: for Go, PHP, and Ruby, "convert custom OTel spans to OneAgent SDK" is not an option — there is no OneAgent SDK for custom tracing in those runtimes. For Python an SDK exists, but Dynatrace's own Python page recommends migrating *from* it to OneAgent auto-instrumentation, so rewriting OTel spans against it runs opposite to the vendor's guidance. The conversation in all four runtimes is "layer OneAgent (where it can run) alongside the existing OTel spans," not "rewrite the OTel spans."

> **Sources:**
>
> - OneAgent SDK for Java (Dynatrace GitHub) — v1.9.0; serverless-not-supported guidance

- OneAgent SDK for .NET (Dynatrace GitHub) — *"the SDK is built using .NET Standard 1.0"*; 1.8.0 is the newest version in both the release-notes and support-status tables, and the metrics surface was withdrawn across two releases — *"1.7.1 Deprecates metrics-related types and APIs"*, then *"1.8.0 Removes deprecated APIs and types."*
- OneAgent SDK for Node.js (Dynatrace GitHub)
- OneAgent SDK for C/C++ (Dynatrace GitHub)
- OneAgent SDK for Python (Dynatrace GitHub) — *"The Dynatrace OneAgent SDK for Python has GA status. The features are fully supported by Dynatrace."*; latest release v1.5.2 (01/07/2026)
- Python technology support (DT docs) — *"we recommend migrating to enable auto-instrumentation using OneAgent for Python"*; CPython 3.8–3.14 listed as Supported
- OneAgent SDK for Go (Dynatrace GitHub) — *"This SDK provides functionality for Go applications to enrich their log messages and OpenTelemetry traces with metadata"*
- OneAgent SDK for PHP (Dynatrace GitHub) — *"providing access to the Trace ID and Span ID information of the PurePath node"*; *"Dynatrace OneAgent SDK for PHP is in Early adopter status."*
- OneAgent SDK Ruby sample binding (Dynatrace GitHub) — archived; *"This project is NOT SUPPORTED"*
- Technology support matrix (DT docs) — Go library tables (gRPC, Kafka, MongoDB, go-redis, pgx, lib/pq, Cassandra, AWS SDK); *"application binary must be dynamically linked unless you're using Go static monitoring"*
- Java technology support (DT docs)
- Go technology support (DT docs) — *"Automatic injection and instrumentation of 64-bit Go executables on x86"*; *"Incoming and outgoing web request monitoring"*
- oneagentctl (DT docs) — connection-parameter changes require *"restart of all the applications monitored with deep code modules"*, because the code module is loaded when the process starts
- Runtime attachment (OpenTelemetry Java contrib GitHub) — *"this project allows you to do attachment programmatically"*; limitation: *"a JDK is necessary"*
- opentelemetry-java-instrumentation (OTel GitHub)
- @opentelemetry/auto-instrumentations-node (OTel JS contrib)
- opentelemetry-python-contrib (OTel GitHub)
- opentelemetry-go-contrib (OTel GitHub)
- opentelemetry-php (OTel GitHub)
- opentelemetry-ruby (OTel GitHub)

4. Capability Comparison

Dimension	OneAgent	OpenTelemetry (auto-instrumentation + SDK)
Owner	Dynatrace	CNCF / OpenTelemetry community (open standard, vendor-neutral)
Backend	Dynatrace — analyzed alongside OTel data from the same services	Any OTLP-compatible backend, Dynatrace included
Runtime model	Native binary on host or in container; injects into supported runtimes (including Go) at startup	Per-runtime: javaagent, .NET injector, Node `--require`, Python `opentelemetry-instrument`, explicit Go imports, PECL extension, Ruby gem
Signals captured	Traces, runtime metrics, host metrics, process metrics, log streams, topology, real-user, deep code-level (PurePath)	Traces, metrics, logs, baggage, context (signals you choose to emit)
Auto-instrumented frameworks	Hundreds across JVM, .NET, Node.js, PHP, Python, plus Go (`net/http`, SQL drivers, gRPC, Kafka, MongoDB, Redis, AWS SDK) — Spring / ASP.NET / Express / Laravel / Django, plus all major SQL drivers, message brokers, web servers, app containers	Per-runtime list — Spring / ASP.NET / Express / Django / Gin / Rails / Laravel etc.; vary in completeness across runtimes (JVM and .NET have the broadest OTel coverage; Go requires explicit imports; Ruby and PHP are still maturing)
Host / runtime / process metrics	Yes — included automatically	Only if you add the host-metrics or runtime-metrics receiver via an OTel Collector / node-exporter; not part of basic SDK

Dimension	OneAgent	OpenTelemetry (auto-instrumentation + SDK)
Smartscape / topology / dependency map	Yes — automatic for hosts, processes, containers and services	Services and service-to-service calls detected from OTel spans; host and process entities with the OpenTelemetry Host Monitoring extension (Collector `hostmetrics`)
Davis AI / problem detection / RCA	Yes — with automatic topology and entity context	Yes — on OTel data ingested into Dynatrace; quality depends on semantic-convention attribute hygiene (`service.name`, resource attributes)
Runtime-access capabilities	Application Security (Runtime Vulnerability Analytics, Runtime Application Protection), Live Debugger (Java, Node.js), automatic RUM JavaScript injection	Not provided by OTel telemetry — these extend an OTel setup rather than compete with it
Serverless — AWS Lambda	**Supported through the Dynatrace AWS Lambda layer** (the OneAgent Lambda extension): tracing for Python, Node.js, Java and .NET; Go is listed as log monitoring only, though OneAgent 1.345 adds monitoring of statically linked Go applications on Lambda. The **OneAgent SDK** is not supported on Lambda — use OTel for custom spans there	Fully supported across all runtimes
Serverless — GCP Cloud Functions	Not supported	Fully supported across all runtimes
Serverless — Azure Functions	**Plan- and runtime-dependent — not a blanket no.** OneAgent 1.343 (released 07/28/2026) adds Azure Functions support for **Python on the Flex Consumption Plan (Linux)** and for **Java and Node.js on Consumption, Premium and Dedicated plans for Windows**; OneAgent 1.345 adds Python triggers (Cosmos DB, Kafka, Queue storage, IoTHub); OneAgent 1.347 (published 10/01/2026, staged rollout from 09/30/2026) adds **Python on the Premium Plan for Linux**. Outside those combinations, still unsupported. Verify the OneAgent version actually deployed on the Function.	Fully supported across all runtimes
Update model	Centrally managed by Dynatrace; no app redeploy	Application redeploy required for SDK; auto-instrumentation update requires process restart
Runtime version floor	Per-runtime; e.g. JVM 8+, .NET Framework 4.5+ / .NET Core 2.1+ / .NET 5+, Node 20+, PHP 7.1+, Python 3.8+ — see Dynatrace supported-technologies matrix for current detail	Per-runtime, and the floors move: Java 8+; all supported .NET and .NET Framework versions except .NET Framework 3.5; Node.js Active/Maintenance LTS (22 / 24 / 26); Python 3.10+; the two newest Go releases (1.26 / 1.27); PHP 8.1+; Ruby 3.3+ — read 10/02/2026, re-check each project before relying on a floor
Code change required	None for auto-coverage	None for auto-instrumentation; explicit imports for custom spans/metrics
Async-context fragmentation risk	Thread-local-style propagation — fragments under fibers (effect-system Scala), some coroutine setups, and async/await without explicit context plumbing	Runtime-aware libraries (otel4s, zio-telemetry, kotlinx-coroutines instrumentation, asyncio context, AsyncLocalStorage, AsyncLocal, context.Context) handle each runtime's async model correctly
Release cadence	Monthly (Dynatrace SaaS sprints)	Per-runtime: monthly minor releases for Java / .NET / Node / Python; Go versions independently

The serverless rule is no longer one rule

"OneAgent does not run on serverless" was never accurate for AWS Lambda, and since OneAgent 1.343 it is not accurate for Azure Functions either. It has to be split by platform — and this matters more here than it would in a reference table, because §9's decision tree treats serverless as a first-question gate, and a wrong answer at a gate misroutes a reader for the life of the project.

- **AWS Lambda** — OneAgent reaches it through the **Dynatrace AWS Lambda layer**, which auto-instruments Python, Node.js, Java and .NET functions without code changes (Go: log monitoring, with statically linked Go monitoring added in OneAgent 1.345). What is *not* supported on Lambda is the **OneAgent SDK** — the per-runtime SDK READMEs send serverless users to OpenTelemetry for custom spans, and for Python, Node.js and Java the Dynatrace Lambda extension can pick up that OTel instrumentation in-process once OpenTelemetry interoperability is enabled (`"OpenTelemetry": { "EnableIntegration": true }` in `dtconfig.json` , per a Dynatrace Classic page last updated 04/2023). That README statement is about the SDK, not the agent, and it is the source of the common misreading that Lambda is OTel-only.
- **GCP Cloud Functions** — OTel is the answer.
- **Azure Functions** — now depends on the **hosting plan** and the **runtime**, not on "is it serverless". OneAgent 1.343 covers Python on the Flex Consumption Plan (Linux), and Java and Node.js on the Consumption, Premium and Dedicated plans for Windows; 1.347 (staged rollout from 09/30/2026) adds Python on the Premium Plan for Linux. Any other plan tier or runtime remains OTel-only.
- **Breaking change in 1.343 — set `DT_WEBSITE_COMPUTE_MODE` where Azure does not.** From 1.343, OneAgent reads `DT_WEBSITE_COMPUTE_MODE` as a fallback when Azure's native `WEBSITE_COMPUTE_MODE` is absent — for example on the Flex Consumption plan or self-deployed Functions — and without it auto-instrumentation may not work. Set it on those Functions before relying on the agent.
- **And still OTel until the agent is actually there.** On the covered Azure combinations, OTel remains the working answer until the OneAgent version deployed on the Function reaches 1.343. **Tenant version is not agent version** — verify the deployed agent rather than inferring support from your tenant's sprint.

The same release also adds OneAgent Azure SDK tracing for **Service Bus** (Java, Node.js, Python), **Event Hub** (Java, Node.js) and **Cosmos DB** (Python); OneAgent 1.347 (staged rollout from 09/30/2026) adds the Cosmos DB client for Node.js. That bears on this decision for a second-order reason: a Function is often thin, and the spans worth having are the messaging and database calls it makes. Where those dependencies were the reason a team reached for OTel, 1.343 changes the calculus even on Function runtimes it does not cover.

> **Sources:**
>
> - OneAgent 1.343 release notes (DT docs) — released 07/28/2026; Azure Functions for Python on the Flex Consumption Plan (Linux) and for Java and Node.js on Consumption, Premium, and Dedicated plans for Windows; Azure SDK tracing for Service Bus, Event Hub and Cosmos DB; breaking change: *"OneAgent reads DT_WEBSITE_COMPUTE_MODE as a fallback when Azure's native WEBSITE_COMPUTE_MODE is absent. Without this fallback variable set, auto-instrumentation may not work in affected setups."*
> - OneAgent 1.345 release notes (DT docs) — *"OneAgent now supports monitoring of statically linked Go applications on AWS Lambda."*; new Python Azure Function triggers (Cosmos DB, Kafka, Queue storage, IoTHub)
> - OneAgent 1.347 release notes (DT docs) — published 10/01/2026, *"Rollout start on Sep 30, 2026"*; *"Extended instrumentation for Azure Function"* for Premium Plan for Linux for Python
> - AWS Lambda integration (DT docs) — *"using auto-instrumentation without code changes"*; *"The OneAgent AWS Lambda extension collects logs directly from Lambda functions"*; per-runtime table lists Python, Node.js, Java and .NET with Lambda layer support and Go as log monitoring only
> - OpenTelemetry on AWS Lambda (DT docs) — Dynatrace Classic, updated 04/24/2023: *"you can use the instrumentation packages available from OpenTelemetry to monitor technologies"*; `EnableIntegration` setting; Python, Node.js and Java
> - OpenTelemetry Java SDK (OpenTelemetry GitHub) — *"Unless otherwise noted, all published artifacts support Java 8 or higher."*; monthly minor releases; latest release v1.66.0 (09/11/2026)
> - OTel floors read 10/02/2026: opentelemetry-dotnet (OpenTelemetry GitHub) — all supported .NET / .NET Framework versions except .NET Framework 3.5; opentelemetry-js (OpenTelemetry GitHub) — *"Only Node.js Active or Maintenance LTS versions are supported."*; opentelemetry-sdk pyproject (OpenTelemetry GitHub) — `requires-python = ">=3.10"` ; opentelemetry-go (OpenTelemetry GitHub) — compatibility table lists Go 1.27 and 1.26; opentelemetry-php composer.json (OpenTelemetry GitHub) — `"php": "^8.1"` ; opentelemetry-sdk gemspec (OpenTelemetry GitHub) — `required_ruby_version = '>= 3.3'`
> - OpenTelemetry main site (opentelemetry.io)
> - Unified services (DT docs) — services detected with Service Detection v2 *"were first introduced for OpenTelemetry"*; *"Automatic endpoint detection and monitoring"*
> - OpenTelemetry Host Monitoring extension (DT docs) — *"This extension automatically generates entities for hosts and their running processes."*
> - Verified on a live tenant 10/13/2026: six services sending only OTLP each had a Smartscape `SERVICE` node, with 9 service-to-service `calls` edges between them and no host, process or Kubernetes edges (`smartscapeNodes "SERVICE" | traverse`).
> - Runtime Vulnerability Analytics (DT docs) — *"Dynatrace OneAgent monitors loaded libraries, runtime components, and data flows in real time."*
> - Runtime Application Protection (DT docs) — *"Code-level visibility provided by OneAgent"*
> - Live Debugger (DT docs) — *"OneAgent version 1.309+ with Java Live-Debugger and Node.js Live-Debugger"*

> - RUM JavaScript injection (DT docs) — *"Once OneAgent is deployed in full-stack monitoring mode on a host, it by default automatically injects the RUM JavaScript"*
> - Use OneAgent with OpenTelemetry (DT docs)
> - OneAgent SDK for Java (Dynatrace GitHub) — *"The OneAgent SDK is not supported on serverless code modules, including those for AWS Lambda."* — the SDK, not the agent
> - **Derived:** the "verify the deployed agent, not the tenant sprint" qualifier follows from agent fleets upgrading independently of tenant version

5. What "Convert to OneAgent" Really Means

When customers say "convert from OTel to OneAgent", they often imagine a 1:1 re-instrumentation effort comparable to the original OTel rollout. **It is not.**

For workloads where OneAgent's automatic coverage is sufficient (the common case — REST/gRPC services on Spring, ASP.NET Core, Express, Django, Rails, plain thread pools or default async runtime), "converting" looks like this:

1. **Install OneAgent** on the host or via the Dynatrace Operator on Kubernetes. Restart the processes. Done — instrumentation is live.
2. **Optionally remove the OTel auto-instrumentation** (the `-javaagent:`, the .NET injector, the Node `--require`, the Python `opentelemetry-instrument`, the explicit Go middleware imports — OneAgent auto-instruments Go `net/http`, SQL drivers, gRPC, Kafka, MongoDB, Redis, Cassandra and AWS SDK clients, per its Go support matrix). This step is *deletion*, not authoring.
3. **Decide what to do with custom OTel spans/metrics/logs** in code:
   - **Easiest path:** leave them in place — Dynatrace ingests OTel data via OTLP, and OneAgent's *OpenTelemetry Span Sensor* (where supported) stitches them into the same trace as OneAgent's auto-instrumented spans. (Caveat: do not enable both Span Sensor and OTLP export simultaneously, or you will get duplicate spans.)
   - **Replacement path:** rewrite each `Tracer` / `Meter` / `Logger` call against the OneAgent SDK for your runtime — *if one with custom tracing exists* (Java, .NET, Node.js, C/C++ and Python — and for Python, Dynatrace recommends OneAgent auto-instrumentation over the SDK). The Go and PHP SDKs only expose trace context for log enrichment and Ruby has no supported SDK, so in those runtimes this path is not available; OTel is the way to emit custom application telemetry. And even where the OneAgent SDK exists, it is intentionally narrow (incoming/outgoing remote calls, custom services, web requests, messaging, SQL, custom request attributes — the Python SDK's feature list is the same shape) and **does not cover metrics or logs at all** — replacing OTel with the OneAgent SDK necessarily *loses* the metrics/logs surface unless you separately ingest them via Dynatrace metrics ingest APIs.

For workloads where OneAgent's automatic coverage is insufficient (async-fragmenting runtimes, exotic frameworks, message brokers OneAgent doesn't auto-instrument), removing OTel is actively harmful — you would be deleting working instrumentation and replacing it with nothing.

The realization: "Convert to OneAgent" is rarely a re-instrumentation project. It is an *agent installation* project, possibly followed by *deleting OTel auto-instrumentation* if you want to simplify the runtime stack. The custom OTel code in the application generally stays exactly where it is.

> **Sources:** Use OneAgent with OpenTelemetry (DT docs) — *"create duplicate spans"* warning; OpenTelemetry Span Sensor mechanism, OneAgent SDK for Java (Dynatrace GitHub), OneAgent SDK for .NET (Dynatrace GitHub) — the release notes record *"1.7.1 Deprecates metrics-related types and APIs"* and then *"1.8.0 Removes deprecated APIs and types."* — leaving the SDK with no metrics or logs surface; OneAgent SDK for Python (Dynatrace GitHub), Python technology support (DT docs) — *"we recommend migrating to enable auto-instrumentation using OneAgent for Python"*, Technology support matrix (DT docs) — Go library tables.

6. Framework Auto-Instrumentation Coverage Across Runtimes

Coverage is the single biggest factor in whether *adding* OneAgent gets you what you want or leaves you patching gaps with the SDK or OTel. The table below summarizes the overlap by runtime — full per-language support matrices link from the `> **Sources:**` block at the end of this section.

Runtime	Web framework / RPC coverage	Database / cache / queue coverage	Notes on gaps
Java / JVM (Java, Kotlin, Scala, Groovy, Clojure)	OneAgent: Spring (MVC + Boot + WebFlux), Servlet / JAX-RS, Akka HTTP 10.1+, Play 2.2–2.8, Tomcat / Jetty / WebLogic / WebSphere / JBoss, gRPC, OkHttp, Apache HttpClient, Java 11+ HttpClient. OTel: similar list plus Akka Actors 2.3+, Finatra 2.9+, Scala ForkJoinPool 2.8+	Both: all major JDBC drivers, Kafka, RabbitMQ / AMQP, JMS, ActiveMQ, Redis, Cassandra, MongoDB, Elasticsearch	Neither auto-instruments **http4s**, **Tapir**, **fs2 streams**, **Monix**, or **Cats Effect IO** — these are async-fragmenting and need runtime-aware OTel libs (see §7)
.NET (C#, F#, VB.NET — CLR + .NET 6 / 7 / 8+)	OneAgent: ASP.NET Framework, ASP.NET Core (Kestrel + IIS), WCF, .NET Remoting, gRPC, HttpClient, MSMQ client. OTel: ASP.NET Core, HttpClient, gRPC, SignalR via `OpenTelemetry.Instrumentation.*` packages	Both: ADO.NET (SQL Server, Oracle, MySQL, Postgres, SQLite), Entity Framework Core, RabbitMQ, Azure Service Bus, Redis (StackExchange.Redis; **ServiceStack.Redis** added in OneAgent 1.343 — verify the agent version deployed), MongoDB	OneAgent auto-instruments classical ASP.NET Framework; OTel coverage of ASP.NET Framework is narrower (mostly ASP.NET Core onwards)
Node.js (JavaScript / TypeScript — V8)	OneAgent: Express, Koa, Hapi, Fastify (newer), NestJS, native HTTP/HTTPS, gRPC. OTel: same list via `@opentelemetry/auto-instrumentations-node` (Express, Koa, Hapi, Fastify, Restify, Connect, GraphQL, gRPC, Apollo)	Both: pg, mysql / mysql2, mongodb, redis, ioredis, kafkajs, amqplib, aws-sdk, elasticsearch	OneAgent and OTel both rely on patching `require()` — module versions outside the supported range are skipped silently
Python (CPython — async-aware)	OneAgent: Django, Flask, FastAPI, Starlette, Tornado, aiohttp server, Bottle, Falcon, Sanic, Azure Functions (versions per the matrix). OTel: similar list via `opentelemetry-instrumentation-django` / `flask` / `fastapi` / `aiohttp` / `tornado` / `pyramid` etc.	Both: psycopg / psycopg2 / asyncpg, PyMySQL / mysqlclient / mysql-connector, MongoDB (pymongo), Redis (redis-py), Kafka (kafka-python / confluent-kafka), Celery, SQLAlchemy	The OneAgent matrix covers the main asyncio frameworks (FastAPI, Starlette, aiohttp, Sanic, Tornado) and `asyncio` itself — compare library versions on both sides rather than assuming either is broader
Go (compiled — OneAgent auto-injects 64-bit executables, 1.323+)	OneAgent: `net/http` (incoming and outgoing), gRPC, custom services, and Go runtime metrics, with no code change; the Span Sensor captures OTel Go spans. OTel: explicit imports — `otelhttp` (net/http), `otelgin`, `otelmux`, `otelchi`, `otelfiber`, `otelgrpc`, `otelecho`	OneAgent: SQL drivers (MySQL, pgx, lib/pq, go-mssqldb), MongoDB, go-redis, Cassandra, DynamoDB; Kafka (sarama, confluent-kafka-go), SQS / SNS / EventBridge. OTel: `otelsql` (database/sql wrapper), `otelpgx`, `otelgorm`, `otelmongo`, `otelredis`, `otelkafka`, `otelsarama`	Check the Go matrix before hand-instrumenting gRPC or Kafka with `otelgrpc` / `otelsarama` — OneAgent already traces the listed versions. Libraries outside the matrix (most ORMs, for example) need OTel. On Linux, statically linked binaries need Go static monitoring. The Go OneAgent SDK has no custom tracing, so custom spans come from OTel — captured by the Span Sensor or sent over OTLP
PHP (Zend Engine 7 / 8)	OneAgent: Apache mod_php / php-fpm, Laravel, Symfony, CodeIgniter, WordPress (with the PHP code module). OTel: `open-telemetry/opentelemetry-auto-instrumentation` Composer packages — Laravel, Symfony, Slim, WordPress, CakePHP	Both: PDO, mysqli, mongodb, Redis, Memcached. OTel additionally: Guzzle HTTP client	OneAgent's PHP coverage is mature; OTel PHP auto-instrumentation is newer and library-by-library
Ruby (MRI, JRuby)	OneAgent: limited Ruby support — primarily host/process telemetry. OTel: `opentelemetry-instrumentation-rails`, `sinatra`, `rack`, `grape`	Both: ActiveRecord, mysql2, pg, redis, mongo. OTel: also `opentelemetry-instrumentation-net_http`, `faraday`	**Ruby is OTel-first** — Dynatrace's recommended path for code-level Ruby tracing is OpenTelemetry, with OneAgent supplying host/process/Smartscape

Patterns to notice:

- **Overlap zone** (most teams gain immediate value from OneAgent) — Java, .NET, Node.js, Python, PHP web stacks: standard frameworks + standard SQL drivers + standard message brokers. OneAgent + OTel duplicates effort here, which is why the §8 Span Sensor / OTLP coexistence pattern exists.

- **OneAgent-only coverage** — on-demand PurePath capture, automatic RUM injection, frontend-backend linking for HTML-document and resource requests (the Server-Timing header; XHR/Fetch requests link from OTel-captured traces too), host/runtime metrics and infrastructure topology without running a Collector, and the runtime-access capabilities (Application Security, Live Debugger). Service topology is not on this list: Dynatrace detects services from OTel spans too.
- **OTel-only coverage** — Ruby application code, Go libraries outside OneAgent's Go support matrix, async-fragmenting runtimes (effect systems, certain coroutine layouts — see §7), GCP Cloud Functions and unsupported Azure Functions plans.
- **Coverage gaps in both** — exotic frameworks (http4s, Tapir, niche message brokers, custom RPC layers). These need either a community OTel instrumentation library, manual instrumentation, or both.

> **Sources:**
>
> - Java technology support (DT docs) — JVM runtime versions
> - Technology support matrix (DT docs) — the OneAgent framework lists per runtime: Akka HTTP server 10.1+, Play Framework 2.2–2.8; .NET Remoting, WCF and MSMQ Client; the Python web frameworks; the Go library tables; *"application binary must be dynamically linked unless you're using Go static monitoring"*
> - Go technology support (DT docs) — *"Automatic injection and instrumentation of 64-bit Go executables on x86"*; *"Incoming and outgoing web request monitoring"*
> - supported-libraries (opentelemetry-java-instrumentation) — kotlinx.coroutines 1.0+, Akka HTTP 10.0+, Akka Actors 2.3+, Finatra 2.9+, Scala ForkJoinPool 2.8+
> - @opentelemetry/auto-instrumentations-node (OTel JS contrib)
> - opentelemetry-python-contrib (OTel GitHub)
> - opentelemetry-go-contrib (OTel GitHub) — `otelhttp` / `otelgin` / `otelmux` / `otelchi` / `otelfiber` / `otelgrpc` / `otelecho` / `otelsql` / `otelpgx` / `otelgorm` / `otelmongo` / `otelredis` / `otelsa`
> - Frontend-backend linking (DT docs) — *"It works for distributed traces captured by OneAgent as well as those captured by OpenTelemetry."*, *"For web frontends with an OTel-instrumented backend, only XHR and Fetch requests can be linked to backend traces."* and *"Requests for the HTML document and its included resources can't be linked, because setting the Server-Timing header is not part of the OpenTelemetry specification."*

7. Async Context Propagation Across Runtimes

Async Context Propagation — Primitives by Runtime

Each runtime carries trace context in a different primitive — does each tool propagate it correctly?

● Propagates correctly ● Mixed / version-dependent ○ Fragments / not applicable

Runtime + async model	Context primitive	OneAgent	OTel (right library)
Java (classical) threads + executors + CompletableFuture	`ThreadLocal`	●	●
Java 21+ (Project Loom) virtual threads on carrier threads	`ScopedValue`	● trace: verify	●
Kotlin coroutines cooperative coroutines on threads	`CoroutineContext`	● 1.10.2–2.1 only	●
Scala (classical) Akka, Play, Future / ForkJoinPool	`ThreadLocal`	●	●
Scala (effect systems) ZIO, Cats Effect, fs2, http4s on IO	`IOLocal / FiberRef`	○ **fragments**	● otel4s / zio-tel
.NET (async / await) Tasks on thread pool	`AsyncLocal<T>`	●	●
Node.js single-threaded event loop	`AsyncLocalStorage / async_hooks`	●	●
Python (asyncio) cooperative coroutines on event loop	`contextvars.ContextVar`	●	●
Go goroutines + explicit context	`context.Context (function arg)`	○ injected; verify	●
PHP request-per-process	`request-scoped globals`	●	●
Ruby threads + fibers (Async gem)	`fiber-local state`	○ no Ruby app injection	●

Fragmenting cells (white, dashed) are where tracing breaks unless you switch to a runtime-aware OTel library.
Scala effect systems are the canonical case: OneAgent fragments, otel4s / zio-telemetry propagate via IOLocal / FiberRef.

This is where the "OneAgent or OTel" choice has a clear-cut technical answer in some runtimes — not a preference.

The mechanic: classical agent-based tracing (both OneAgent's auto-instrumentation and OTel's auto-instrumentation in most runtimes) propagates trace context using a runtime thread-local construct. As long as a request stays on a single thread (or the agent has hooked the thread-pool / scheduling primitives the application uses), context flows correctly. Some runtimes break this assumption.

Per-runtime async-context primitives — and how each tool handles them:

Runtime	Async / concurrency model	Context primitive	OneAgent	OTel (with right library)
Java (classical)	OS threads + thread pools + `CompletableFuture`	`ThreadLocal`	Yes — auto-instruments common executors	Yes — Java agent hooks executors
Java 21+ (Project Loom virtual threads)	Many lightweight virtual threads on few carrier threads	`ScopedValue` / virtual-thread-local	Verify — the Java page documents CPU profiling *"including support for virtual threads"*; trace-context propagation across virtual threads is not stated	Yes — current OTel Java agent supports virtual threads
Kotlin coroutines	Cooperative coroutines on threads	`CoroutineContext`	Yes for Kotlin Coroutines 1.10.2–2.1 (support matrix); verify other versions	Yes — `kotlinx.coroutines` OTel instrumentation
Scala — classical (Akka, Play, plain `Future` / `ForkJoinPool`)	OS threads / executor service	`ThreadLocal`	Yes — strong auto-instrumentation	Yes — OTel Java agent
Scala — effect systems (ZIO, Cats Effect, fs2, http4s on IO)	Lightweight fibers multiplexed onto a small thread pool; runtime explicitly does *not* preserve `ThreadLocal` across fiber suspension	`IOLocal` (Cats Effect) / `FiberRef` (ZIO)	**No** — fragments across fiber boundaries	**Yes** — *otel4s* uses `IOLocal`; *zio-telemetry* uses `FiberRef`; both propagate correctly
.NET (`async` / `await`)	Tasks scheduled on a thread pool with logical-call-context flow	`AsyncLocal<T>`	Yes — OneAgent flows context through `AsyncLocal`; .NET runtime-async from OneAgent 1.345, with the Advanced .NET Instrumentation Mode feature enabled	Yes — `System.Diagnostics.Activity` uses `AsyncLocal` natively
Node.js	Single-threaded event loop with callback / Promise chains	`AsyncLocalStorage` (built on `async_hooks`)	Yes — OneAgent's Node module hooks `async_hooks`	Yes — `@opentelemetry/context-async-hooks`
Python (asyncio)	Cooperative coroutines on an event loop	`contextvars.ContextVar`	Yes (recent Python code module)	Yes — `opentelemetry-api` uses `contextvars`
Python (greenlet / gevent / eventlet)	Cooperative greenlets monkey-patching the stdlib	Library-specific	Mixed — Gevent 20.9.0+ is in the OneAgent matrix; eventlet is not	Mixed — OTel has per-library shims
Go	Goroutines with explicit `context.Context` plumbing	`context.Context` (passed as a function argument by convention)	OneAgent injects Go (`net/http` , SQL, gRPC, Kafka and more — §6); verify propagation across your goroutine layout	Yes — every OTel Go library takes a `context.Context`

Runtime	Async / concurrency model	Context primitive	OneAgent	OTel (with right library)
PHP	Synchronous request-per-process model (mostly)	Request-scoped globals	Yes — OneAgent's PHP module covers it	Yes — OTel PHP covers it
Ruby	OS threads + (in newer Rubies) fibers; `Async` gem and ractors are evolving	Fiber-local state	N/A in code (Dynatrace recommends OTel for Ruby app code)	Yes — `OpenTelemetry::Context`

Symptoms when context propagation breaks (in any runtime):

- A trace begins on a request, then "loses" context after the first asynchronous boundary
- Spans appear as orphans (no parent) downstream of the boundary
- Database calls, HTTP calls, and message publishes inside an async block do not link back to the parent request span
- The trace waterfall looks plausibly populated but is structurally wrong

Practical guidance by runtime:

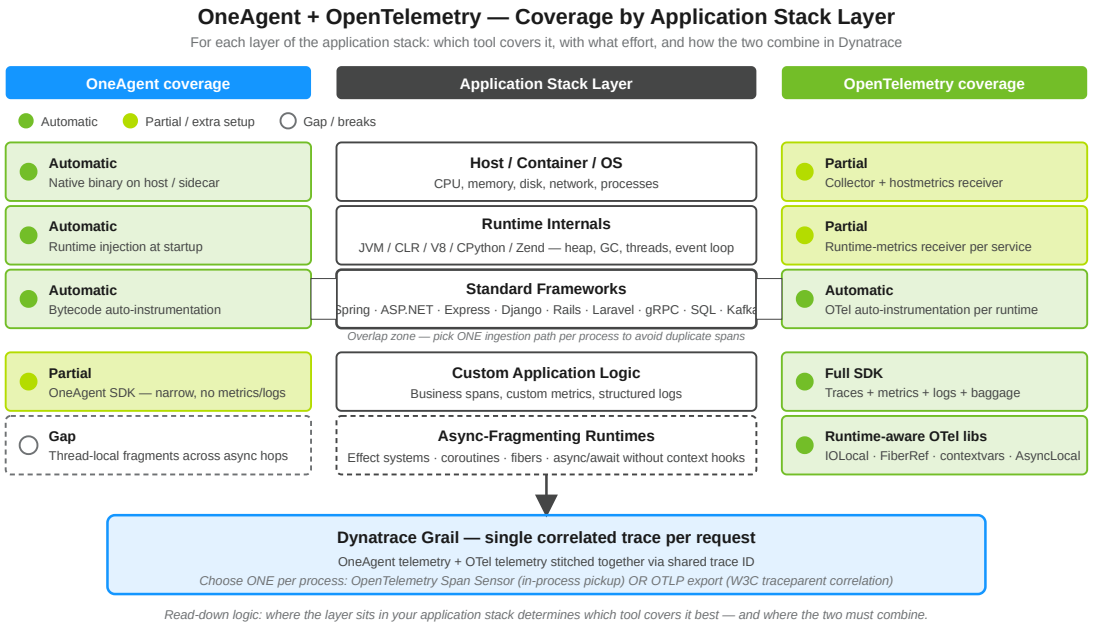
Runtime + scenario	App-level tracing	Infra-level coverage
Java / .NET / Node / Python — standard frameworks on default scheduler	OneAgent auto-instrumentation (no code change)	OneAgent (same)
Java with virtual threads	OTel Java agent; OneAgent — verify trace propagation on your agent version (documented for profiling)	OneAgent
Kotlin coroutines	OTel `kotlinx.coroutines` instrumentation if OneAgent's coroutine support is incomplete in your environment	OneAgent
Scala — Akka, Play, plain `Future` / `ForkJoinPool`	OneAgent auto-instrumentation	OneAgent
Scala — ZIO + zio-http / sttp on ZIO	**zio-telemetry (OTel)** — OneAgent fragments; SDK has no `FiberRef` model	OneAgent for host/runtime
Scala — Cats Effect + http4s + fs2	**otel4s (OTel)** — OneAgent fragments; SDK has no `IOLocal` model	OneAgent for host/runtime
Mixed Scala estate (some Akka, some ZIO)	Per-service: OneAgent for Akka/Play, OTel for effect systems	OneAgent everywhere
Go	OneAgent auto-injection for the libraries in its Go matrix (`net/http` , SQL drivers, gRPC, Kafka, MongoDB, Redis, Cassandra and AWS SDK clients); **OTel** for other libraries and custom spans (the Go OneAgent SDK has no custom tracing)	OneAgent
Ruby	**OTel only** at app level — Dynatrace's documented recommendation	OneAgent for host/process
AWS Lambda	Dynatrace Lambda layer (Python, Node.js, Java, .NET); **OTel** for custom spans — the OneAgent SDK is not supported there	—
GCP Cloud Functions, unsupported Azure Functions plans	**OTel only**	—

The principle: if the runtime has an async model that fragments thread-local storage (effect systems, certain coroutine layouts, or any runtime where context is fiber-scoped or scope-of-task), use the OTel library that targets that runtime's context primitive. The OneAgent SDK generally has a thread-local-style context model and will fragment in the same way unless the agent itself has been updated to hook the runtime's async primitives. The documented exception is the Python SDK running under the OneAgent Python code module, whose current tracer is context-local through `contextvars` .

Sources:

- AsyncLocal<T> (Microsoft Learn) — *"ambient data local to a given asynchronous control flow"*
- AsyncLocalStorage (Node.js docs) — built on `node:async_hooks`
- contextvars (Python docs) — natively supported in `asyncio` (Python 3.7+)
- otel4s (Typelevel GitHub) — *"design goal is to fully and faithfully implement the OpenTelemetry Specification atop Cats Effect"*
- zio-telemetry (ZIO GitHub)
- kotlinx-coroutines instrumentation (OTel Java)
- supported-libraries (opentelemetry-java-instrumentation)
- Go OpenTelemetry walkthrough (DT docs)
- Technology support matrix (DT docs) — *"Kotlin Coroutines 1.10.2 - 2.1"*; Gevent 20.9.0+ and `asyncio` in the Python table
- Java technology support (DT docs) — *"production-grade CPU profiling (including support for virtual threads)"*; the page does not mention virtual threads for trace propagation
- OneAgent 1.345 release notes (DT docs) — *"OneAgent now supports in-process context propagation and span tracing when using .NET runtime-async. This requires the Advanced .NET Instrumentation Mode OneAgent feature to be enabled."*
- OneAgent SDK for Python (Dynatrace GitHub) — with the OneAgent code module for Python the current tracer *"works as expected with both multithreading"* and `asyncio`, using `contextvars`

8. The Coexistence Pattern (How They Work Together in Dynatrace)



Dynatrace is explicit that the two are designed to coexist. From the Dynatrace OneAgent + OpenTelemetry documentation: *"You can implement this service by service, adopting the open standards of OpenTelemetry where openness matters most, while leveraging enhanced OneAgent features available where you need them."*

The mechanism — OpenTelemetry Span Sensor: when this code-module setting is enabled (where supported per runtime), OneAgent detects in-process OpenTelemetry API calls and weaves them into the same trace as OneAgent's auto-instrumented spans. Per the Dynatrace docs: *"These auto-instrumented spans are woven together with your manual OpenTelemetry spans into a single trace."* The result is a unified PurePath-style trace that contains both OneAgent and OTel-emitted spans, correlated through a shared trace ID.

Critical caveat: if you enable the OpenTelemetry Span Sensor *and* simultaneously export OTLP from the same process, **you will get duplicate spans** — once from the Span Sensor's in-process pickup, once from the OTLP exporter. Pick one ingestion path per process. The two supported patterns are:

1. **Span Sensor only** — OTel API calls in code, no OTLP exporter configured. OneAgent picks them up locally, ships them inside its native channel.

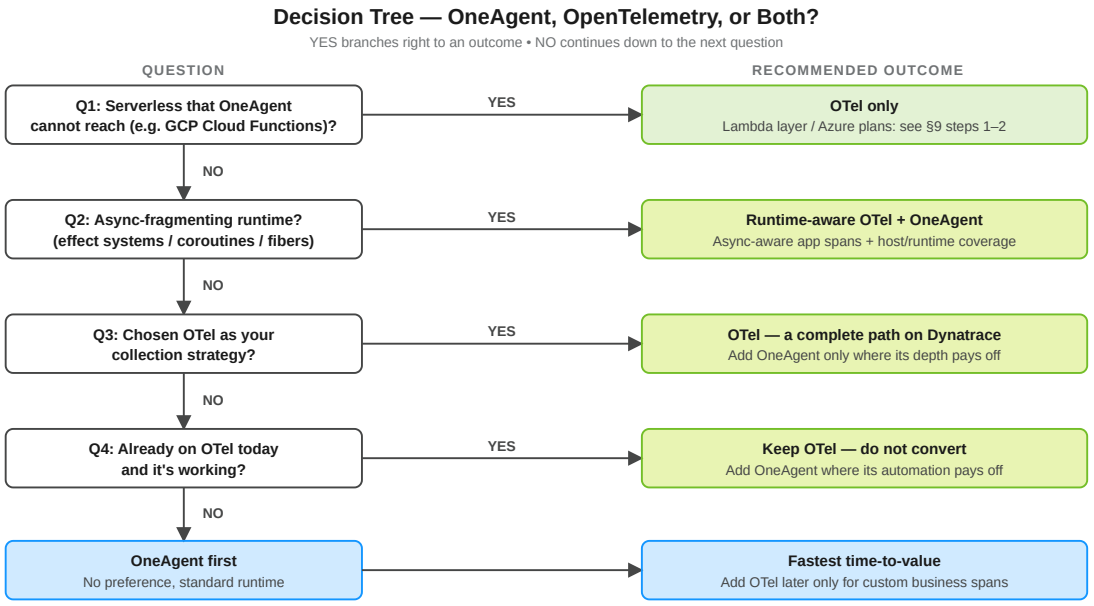
2. **OTLP only** — Span Sensor disabled, OTel exporter configured to send to a Dynatrace OTLP endpoint (or to an OTel Collector forwarding to Dynatrace). OneAgent traces and OTel traces correlate via standard W3C `traceparent` propagation.

Pattern 1 is documented to weave the OTel spans into the OneAgent trace; with Pattern 2, correlation relies on W3C `traceparent` propagation — verify the resulting trace structure in your environment. Two things to know before choosing Pattern 1: the docs describe the Span Sensor as *"for advanced users only"*, and on Java, enabling it replaces the application's existing tracers — if the application also exports OTel to another backend, that export stops, so use Pattern 2 there. For runtimes without Span Sensor support (Ruby, and Python where your tenant has no Python switch — see below), for platforms OneAgent does not reach (GCP Cloud Functions, unsupported Azure Functions plans), and for any process without OneAgent, Pattern 2 (OTLP) is the only option.

Span Sensor runtime support: the *Use OneAgent with OpenTelemetry* page lists **Java, Go, Node.js, PHP, and .NET** — so Go is a Span Sensor runtime, not an OTLP-only one. The *Enable the OpenTelemetry Span Sensor* configuration page additionally documents an opt-in **OpenTelemetry (Python)** switch, and OneAgent 1.347 fixes span-link IDs for OpenTelemetry Python spans captured by OneAgent. The two Dynatrace pages disagree on Python: check for the switch under your tenant's OneAgent features before planning a Python service around Pattern 1.

> **Sources:**
>
> - Use OneAgent with OpenTelemetry (DT docs) — Span Sensor mechanism, *"woven together .. single trace"*, duplicate-spans warning, *"This feature is for advanced users only"*, *"OpenTelemetry span data can be captured for Java, Go, Node.js, PHP, and .NET"*
> - Enable the OpenTelemetry Span Sensor for OneAgent (DT docs) — Java: *"Existing tracers are replaced and will no longer work after you enable OpenTelemetry Java."*; Python: *"Find and enable OpenTelemetry (Python) [Opt-In]"*, *"Existing tracers are not affected by OneAgent OpenTelemetry for Python support."*
> - OneAgent 1.347 release notes (DT docs) — *"Fixed incorrect span and trace IDs in span links created with OpenTelemetry Python when captured by OneAgent."*
> - Extend Dynatrace with OpenTelemetry (DT docs) — *"adopting the open standards of OpenTelemetry where openness matters most"*
> - Ingest OpenTelemetry — getting started (DT docs) — OTLP endpoints, Collector, Istio/Envoy paths
> - W3C Trace Context (opentelemetry-specification) — `traceparent` propagation

9. Decision Tree by Workload Type



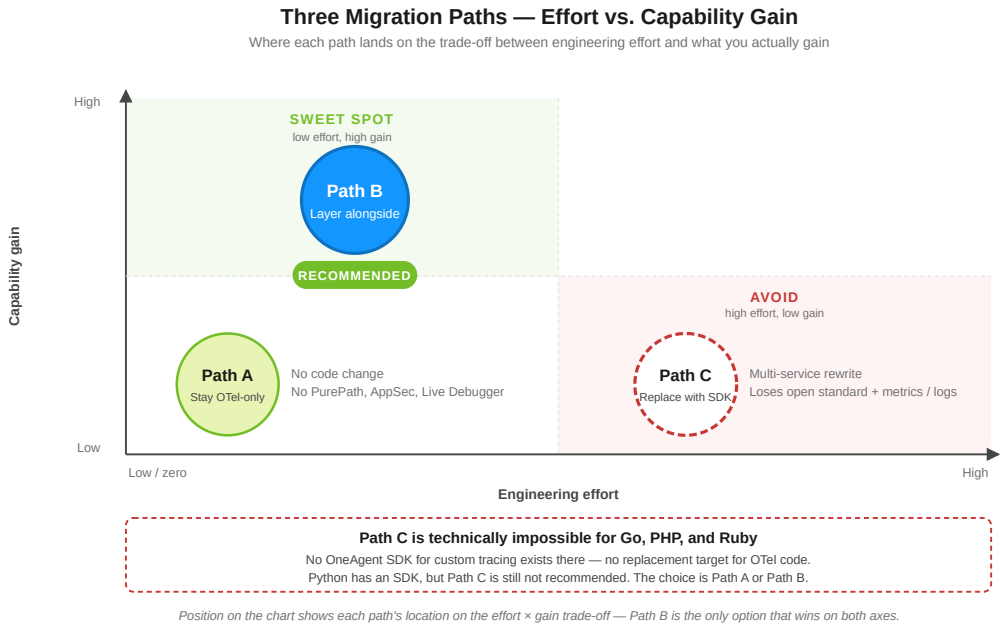
Plain-language flow:

1. **AWS Lambda or GCP Cloud Functions?** → Split them. **GCP Cloud Functions:** OTel only. **AWS Lambda:** the Dynatrace Lambda layer auto-instruments Python, Node.js, Java and .NET functions; use OTel for custom spans, because the *OneAgent SDK* — not OneAgent — is unsupported on Lambda (the per-runtime SDK READMEs direct serverless users to OpenTelemetry for that reason). Go on Lambda is log monitoring, with statically linked Go monitoring added in OneAgent 1.345.

2. **Azure Functions? → Ask two more questions, not one.** As of **OneAgent 1.343 (released 07/28/2026)** Azure Functions is no longer a blanket no: **Python on the Flex Consumption Plan (Linux)** and **Java and Node.js on Consumption, Premium and Dedicated plans for Windows** are supported, across multiple trigger types — and 1.347 (staged rollout from 09/30/2026) adds **Python on the Premium Plan for Linux**. So the gate is *plan* and *runtime*, not "is it serverless."
   - Outside those combinations → OTel, as before.
   - Inside them → OneAgent is an option, **once the OneAgent version deployed on the Function actually reaches 1.343.** Tenant version is not agent version; verify the deployed agent. Until it does, OTel remains the working path. Where Azure does not set `WEBSITE_COMPUTE_MODE` (Flex Consumption, self-deployed Functions), set `DT_WEBSITE_COMPUTE_MODE`, or auto-instrumentation may not work (§4).
   - Either way, note that 1.343 also adds OneAgent Azure SDK tracing for **Service Bus** (Java, Node.js, Python), **Event Hub** (Java, Node.js) and **Cosmos DB** (Python). If the reason you wanted tracing was the Function's *dependencies* rather than the Function itself, that may be the more consequential half of the release for you.
3. **Async-fragmenting runtime?** (effect systems, coroutines without agent hooks, async without explicit context propagation) → OTel via the runtime-aware library — *otel4s* / *zio-telemetry* (Scala effect systems), `kotlinx.coroutines` OTel instrumentation (Kotlin), `contextvars` (Python asyncio), `AsyncLocal` (.NET — handled natively), `AsyncLocalStorage` (Node.js — handled natively), `context.Context` (Go — handled natively by the OTel Go libraries). Add OneAgent for host/runtime signals if the workload runs on long-lived hosts or containers.
4. **Chosen OTel as your collection strategy** (including an open-standard or multi-backend mandate)? → OTel is a complete path on Dynatrace — services, endpoints and service topology are detected from OTel data, and the OpenTelemetry Host Monitoring extension adds host and process entities. Add OneAgent only where its fully managed collection or runtime-access capabilities pay off for a workload.
5. **Runtime is Ruby?** → OTel for application spans; OneAgent adds host/process telemetry and Smartscape. **Runtime is Go?** → OneAgent auto-instruments `net/http`, SQL drivers, gRPC, Kafka, MongoDB, Redis, Cassandra and AWS SDK clients (check the Go support matrix); OTel covers other libraries and custom spans (the Go OneAgent SDK has no custom tracing), captured by the Span Sensor or sent over OTLP.
6. **Standard web stack?** (Spring, ASP.NET Core, Express, Fastify, Django, FastAPI, Rails, Laravel, etc.) → OneAgent's auto-instrumentation gives you the fastest time-to-value. If you already have OTel, keep it for custom spans; if not, you may not need it at all.
7. **Already instrumented with OTel and it's working?** → Keep it; OTel-only is a complete path. Add OneAgent alongside where its automation or runtime-access capabilities pay off, and do *not* rip out OTel. The combined cost is lower than the conversion cost, and you keep your open-standard instrumentation.

> **Note on the diagram.** The SVG compresses the serverless branch into one gate for platforms OneAgent does not reach. Read steps 1 and 2 as the authoritative form of that branch.
>
> **Sources:**
>
> - OneAgent 1.343 release notes (DT docs) — released 07/28/2026; Azure Functions support for Python on the Flex Consumption Plan (Linux) and Java / Node.js on Windows plans; Azure Service Bus / Event Hub / Cosmos DB auto-tracing
> - OneAgent SDK for Java (Dynatrace GitHub) — *"not supported on serverless code modules .. Consider using OpenTelemetry instead"*
> - OneAgent SDK for Node.js (Dynatrace GitHub)
> - OpenTelemetry on AWS Lambda (DT docs) — Lambda extension + OTel; Python, Node, Java
> - AWS Lambda integration (DT docs) — *"using auto-instrumentation without code changes"*
> - Use OneAgent with OpenTelemetry (DT docs)
> - Technology support matrix (DT docs) — Go library tables
> - OneAgent 1.347 release notes (DT docs) — *"Rollout start on Sep 30, 2026"*
> - otel4s (Typelevel GitHub), zio-telemetry (ZIO GitHub)

10. Three Migration Paths and Their Effort



Position on the chart shows each path's location on the effort × gain trade-off — Path B is the only option that wins on both axes.

Path A: Stay OTel-only (no OneAgent at all)

Aspect	Reality
Effort	Zero — keep current state
Code changes	None
What you keep	Open-standard instrumentation, runtime-aware tracing on async-fragmenting stacks, serverless coverage, single instrumentation model across polyglot stacks
What you give up	Infrastructure topology and automatic runtime/host/process metrics without running a Collector, on-demand code-level method tracing, Application Security and Live Debugger, automatic RUM injection, and frontend-backend linking for HTML-document and resource requests (XHR/Fetch requests still link through W3C trace context)
Best for	Organizations that have chosen OTel as their collection strategy; greenfield CNCF-aligned organizations; serverless-heavy estates; multi-cloud/multi-vendor strategies; Go/Ruby-dominant or effect-system-dominant teams

Path B: Add OneAgent alongside OTel (the layered pattern — recommended for most)

Aspect	Reality
Effort	Low — install OneAgent on hosts or via Dynatrace Operator, restart the application processes
Code changes	None — keep all existing OTel code
What you gain	Fully managed collection, host/runtime metrics and infrastructure topology without a Collector, Application Security, Live Debugger, automatic framework coverage where it overlaps with OTel, RUM injection and HTML-document linking
What you must decide	Span Sensor vs OTLP export (pick one per process — see §8 caveat)
Risk	Duplicate spans if both ingestion paths active simultaneously; resolved by configuration
Best for	Teams already on OTel who want OneAgent's automation and runtime depth without giving up their open-standard instrumentation

Path C: Replace OTel instrumentation with OneAgent (and the OneAgent SDK for custom spans)

Aspect	Reality
Effort	Highest — install OneAgent, remove OTel auto-instrumentation + dependencies, rewrite every custom span / metric / log against the OneAgent SDK
Code changes	Per-service rewrite of every `Tracer.spanBuilder(...)` / `ActivitySource.StartActivity(...)` / equivalent, plus every metric and log emission to OneAgent SDK equivalents — and the SDK does not cover metrics or logs at all, so those signals are lost or must be re-emitted via Dynatrace metrics ingest APIs
Availability	OneAgent SDKs with custom tracing exist for Java, .NET, Node.js, C/C++ and Python. **Path C is not an option for Go, PHP or Ruby** — the Go and PHP SDKs only expose trace context for log enrichment, and Ruby has no supported SDK. For Python it is possible, but Dynatrace recommends migrating *from* the Python SDK to OneAgent auto-instrumentation
What you gain	One instrumentation model in code; tighter native PurePath semantics in places
What you lose	Open-standard instrumentation, broad community ecosystem, runtime-aware libraries on async-fragmenting stacks, serverless coverage, parts of your metrics/logs surface
Risk	High — coordinated multi-service rewrite, regression-prone, must be repeated for every new service
Best for	Almost no one. Defensible only if (a) the engineering org has a strict single-vendor mandate, (b) all workloads are non-serverless and on a runtime where the OneAgent SDK exists, (c) no async-fragmenting runtimes are involved, and (d) the cost of dual maintenance demonstrably exceeds the rewrite cost

> **Sources:** OneAgent SDK for Java (Dynatrace GitHub), OneAgent SDK for .NET (Dynatrace GitHub) — *"Removes deprecated APIs"* (v1.8.0); no metrics/logs surface, OneAgent SDK for Python (Dynatrace GitHub), OneAgent SDK for Go (Dynatrace GitHub), OneAgent SDK for PHP (Dynatrace GitHub), Python technology support (DT docs) — *"we recommend migrating to enable auto-instrumentation using OneAgent for Python"*, Use OneAgent with OpenTelemetry (DT docs), Runtime Vulnerability Analytics (DT docs), Live Debugger (DT docs), Frontend-backend linking (DT docs) — *"For web frontends with an OTel-instrumented backend, only XHR and Fetch requests can be linked to backend traces."*

11. Pros and Cons Side-by-Side

OneAgent — pros

- Zero-code instrumentation for hundreds of frameworks across Java, .NET, Node.js, Python, PHP, plus Go (`net/http`, SQL drivers, gRPC, Kafka and more)
- Host, runtime, process, network, and topology coverage out of the box
- Smartscape dependency map for hosts, processes, containers and services, filled in automatically
- Runtime-access capabilities OTel telemetry does not provide: Application Security (Runtime Vulnerability Analytics, Runtime Application Protection), Live Debugger, automatic RUM JavaScript injection
- Frontend-backend linking for HTML-document and resource requests, through the Server-Timing header OneAgent adds (XHR/Fetch requests link from OTel-captured traces too)
- Centrally managed updates — no application redeploy
- Operations-team-friendly (instrument an environment without engineering involvement)

OneAgent — cons

- Does not run on GCP Cloud Functions or unsupported Azure Functions plans; on AWS Lambda it runs as the Dynatrace Lambda layer, but the OneAgent SDK does not
- Does not inject Ruby application code (still adds host/process value); Go coverage is limited to the libraries in its Go support matrix, and statically linked Linux binaries need Go static monitoring
- Default thread-local-style context model fragments under some async runtimes (effect systems, certain coroutine layouts)
- Custom-span surface (OneAgent SDK) is intentionally narrow — no metrics or log signals; no custom-tracing SDK for Go, PHP or Ruby
- Framework support is broad but not exhaustive (e.g., http4s, Tapir, niche message brokers, exotic RPC layers)

OpenTelemetry — pros

- Open standard and vendor-neutral — one instrumentation model and common semantic conventions, exportable to any OTLP-compatible backend, Dynatrace included
- Full signal coverage — traces, metrics, logs, baggage
- Per-runtime context-aware libraries (otel4s, zio-telemetry, kotlinx-coroutines, contextvars, context.Context, AsyncLocal, AsyncLocalStorage) handle each runtime's async model correctly
- Services, endpoints and service topology detected from OTel spans in Dynatrace
- First-class on serverless across every major runtime
- First-class on Ruby, where OneAgent doesn't inject application code, and on Go libraries outside OneAgent's Go support matrix
- Strong community ecosystem, monthly releases, broad plugin coverage
- Code-level control of sampling, attributes, and span structure

OpenTelemetry — cons

- Host and runtime coverage means operating Collectors and receivers; host and process entities need the OpenTelemetry Host Monitoring extension
- No equivalent to on-demand PurePath method tracing
- Requires application redeploy for SDK changes
- Analysis quality depends on consistent semantic-convention attributes, which the instrumenting team owns
- More moving parts (auto-instrumentation, SDK code, Collector) than a single OneAgent install
- Coverage breadth varies per runtime — JVM and .NET have the deepest OTel coverage; Ruby and PHP are still maturing

> **Sources:**
>
> ˜ Technology support matrix (DT docs) — OneAgent framework, host, runtime coverage
>
> ˜ Frontend-backend linking (DT docs) — *"It works for distributed traces captured by OneAgent as well as those captured by OpenTelemetry."*, *"For web frontends with an OTel-instrumented backend, only XHR and Fetch requests can be linked to backend traces."* and *"Requests for the HTML document and its included resources can't be linked, because setting the Server-Timing header is not part of the OpenTelemetry specification."*
>
> ˜ Use OneAgent with OpenTelemetry (DT docs)
>
> ˜ **Derived:** Smartscape and PurePath as OneAgent-dependent capabilities combines the §2 capability table with the technology-support matrix; the coexistence page does not state it
>
> ˜ Runtime Vulnerability Analytics (DT docs) — *"Dynatrace OneAgent monitors loaded libraries, runtime components, and data flows in real time."*
>
> ˜ Runtime Application Protection (DT docs) — *"Code-level visibility provided by OneAgent"*
>
> ˜ Live Debugger (DT docs) — *"OneAgent version 1.309+ with Java Live-Debugger and Node.js Live-Debugger"*
>
> ˜ RUM JavaScript injection (DT docs) — *"Once OneAgent is deployed in full-stack monitoring mode on a host, it by default automatically injects the RUM JavaScript"*
>
> ˜ OpenTelemetry main site (opentelemetry.io) — vendor neutrality, signal coverage, ecosystem
>
> ˜ Unified services (DT docs) — services detected with Service Detection v2 *"were first introduced for OpenTelemetry"*; *"Automatic endpoint detection and monitoring"*
>
> ˜ OpenTelemetry Host Monitoring extension (DT docs) — *"This extension automatically generates entities for hosts and their running processes."*
>
> ˜ Verified on a live tenant 10/13/2026: six services sending only OTLP each had a Smartscape `SERVICE` node, with 9 service-to-service `calls` edges between them and no host, process or Kubernetes edges (`smartscapeNodes "SERVICE" | traverse`).
>
> ˜ OpenTelemetry Java SDK (OpenTelemetry GitHub) — monthly minor releases

12. Is the Conversion Worth the Effort?

Question every customer should ask before starting: *"What capability am I gaining that I do not already have, and is it worth the engineering disruption to get it?"*

Run through this checklist honestly:

1. **Does my OTel data already land in Dynatrace via OTLP?** If yes, you already have most of what "converting" would give you for traces.
2. **Am I missing host, runtime, or infrastructure-topology signals?** If yes, either add OneAgent or extend the Collector (`hostmetrics` plus the OpenTelemetry Host Monitoring extension) — *removing* OTel solves nothing.

3. **Am I on an async-fragmenting runtime?** (Scala effect systems, certain coroutine layouts.) If yes, removing the runtime-aware OTel layer will *break* tracing — there is no OneAgent equivalent.
4. **Am I on Go, PHP or Ruby?** If yes, there is no OneAgent SDK for custom tracing to replace OTel with. Conversion is not technically possible at the application-span level (OneAgent does auto-instrument the Go libraries in its support matrix, so Go keeps that layer either way). On Python an SDK exists, but Dynatrace recommends OneAgent auto-instrumentation over it.
5. **Do I have serverless workloads OneAgent does not reach?** GCP Cloud Functions and unsupported Azure Functions plans need OTel, and on AWS Lambda custom spans need OTel because the OneAgent SDK is unsupported there — so a hybrid is unavoidable.
6. **Is there an organizational mandate for open-standard instrumentation or multi-backend export?** If yes, OTel owns application instrumentation — and OneAgent can still run alongside it.
7. **What is the cost of a coordinated multi-service rewrite of custom spans, metrics, and logs?** Compare honestly against the cost of running both for the foreseeable future.
8. **Is anyone on the team going to maintain a parallel internal instrumentation library against the OneAgent SDK?** If not, even partial Path C is a slow walk into instrumentation rot.

Common false economies:

- *"Removing OTel will reduce overhead."* In community practice, OTel auto-instrumentation overhead is small enough that removing it after installing OneAgent does not measurably move the needle. OneAgent has its own overhead in the same general range. Exact percentages depend heavily on workload, sampling rate, and exporter configuration — measure in your environment rather than relying on a published number.
- *"One instrumentation model is simpler."* True at the code level, false at the platform level — Dynatrace was designed to merge both, and customers running both report fewer surprises than customers running one tool stretched beyond its design center.
- *"OneAgent traces are better than OTel traces."* For overlapping framework coverage on standard stacks, the data is comparable. OneAgent's advantage is in what surrounds the spans — host, runtime and topology context, plus runtime-access capabilities such as Application Security and Live Debugger.

The honest answer for almost every team already on OTel: the level of effort to *add* OneAgent is small (a deployment-engineering task, not a re-instrumentation task). The level of effort to *replace* OTel with OneAgent is large — and impossible in some runtimes — and almost never repaid by the marginal capability gained. **Layer, don't convert.**

> **Sources:** OneAgent SDK for Java (Dynatrace GitHub) — what the SDK does and does not cover, OneAgent SDK for .NET (Dynatrace GitHub) — metrics/logs out of scope, Use OneAgent with OpenTelemetry (DT docs), OneAgent SDK for Go (Dynatrace GitHub) — enrichment metadata only, OneAgent SDK for PHP (Dynatrace GitHub) — Trace ID / Span ID only.
>
> The §12 overhead range and dual-tooling observation are framed as **community guidance** — vendor-published numbers were not located. Measure overhead in your own environment.

13. Recommended Approach

Default recommendation for any backend already instrumented with OpenTelemetry:

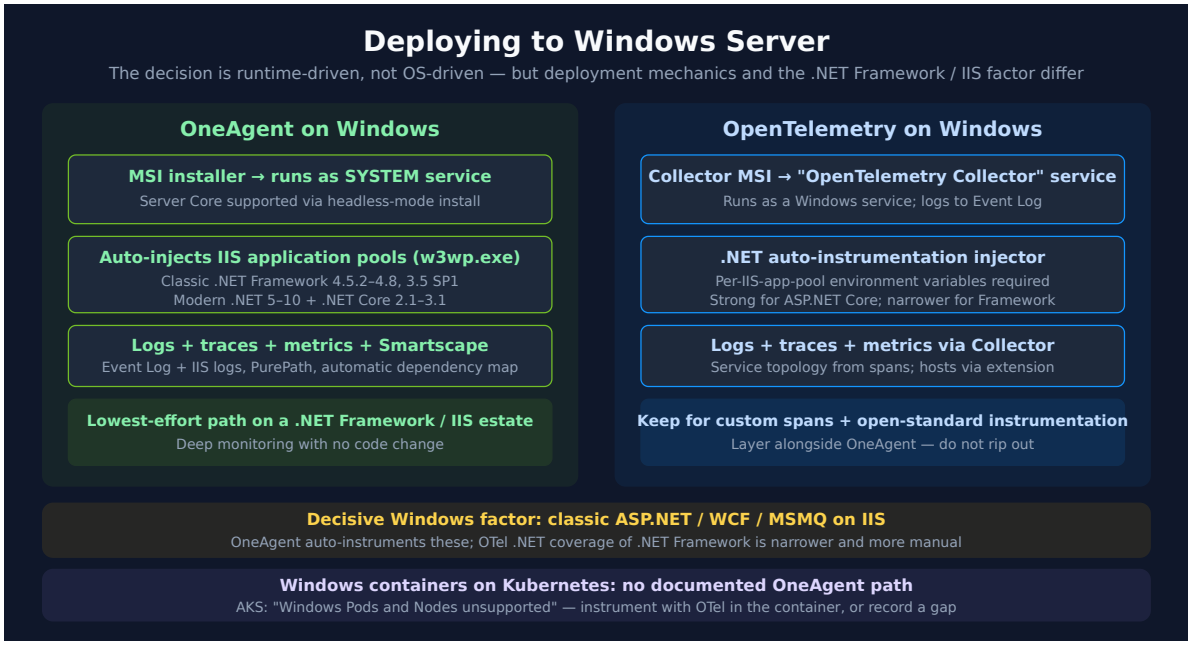
1. **Keep your OpenTelemetry instrumentation in place.** Do not start a conversion project.
2. **Decide per workload whether OneAgent adds enough.** Where OTel is your strategy, OTel-only is complete — run the Collector with `hostmetrics` and activate the OpenTelemetry Host Monitoring extension for host and process entities. Where you want fully managed collection or runtime-access capabilities, **install OneAgent** on the hosts (or via the Dynatrace Operator on Kubernetes) where the services run, and restart the processes. (For Ruby workloads, OneAgent still adds host/process/Smartscape value even though it doesn't inject application code. For Go, it also auto-instruments `net/http`, SQL drivers, gRPC, Kafka, MongoDB, Redis, Cassandra and AWS SDK clients per its Go support matrix; on Linux, statically linked binaries need Go static monitoring.)
3. **Pick one of the two supported coexistence patterns** to avoid duplicate spans:
   - *Pattern 1 — Span Sensor:* enable the OpenTelemetry Span Sensor in the OneAgent code module for your runtime; **disable** any OTLP exporter in the application. OneAgent picks up the OTel API calls in-process. (Available where the OneAgent code module supports it — check per-runtime docs. The docs describe it as *for advanced users only*, and on Java enabling it replaces the application's existing tracers — if the app also exports OTel to another backend, use Pattern 2.)
   - *Pattern 2 — OTLP:* keep the OTel exporter pointed at Dynatrace's OTLP endpoint (or an OTel Collector forwarding to Dynatrace); **disable** the Span Sensor. Spans correlate via W3C `traceparent`. This is the only option for Ruby (no Span Sensor), for Python where your tenant has no Python Span Sensor switch (§8), for GCP Cloud Functions, and for any process without OneAgent.

4. **For async-fragmenting runtimes** (Scala effect systems, certain coroutine layouts), ensure the OTel layer is the runtime-aware library (otel4s, zio-telemetry, kotlinx-coroutines instrumentation) rather than the raw default SDK. Do not attempt to replace this layer with the OneAgent SDK — even where the SDK exists, it has no equivalent of the runtime's async-context primitive.
5. **For serverless workloads**, split by platform: AWS Lambda — the Dynatrace Lambda layer plus OTel for custom spans; supported Azure Functions plans — OneAgent once the deployed agent reaches 1.343; GCP Cloud Functions and anything else — OTel only.
6. **Do not rewrite custom OTel spans, metrics, or logs against the OneAgent SDK** unless you have a specific Dynatrace capability that the SDK uniquely enables (in practice, this is rarely the case — and there is no custom-tracing SDK for Go, PHP or Ruby, while for Python Dynatrace recommends OneAgent auto-instrumentation over its SDK).
7. **Revisit per workload** — as OneAgent coverage grows in your runtimes (new async-aware tracing, new frameworks), more of the stack can move to automatic instrumentation while OTel keeps owning custom spans. Today, layering is the right answer wherever OneAgent runs and its depth is wanted.

> **Sources:**
>
> - Use OneAgent with OpenTelemetry (DT docs) — Span Sensor and OTLP coexistence patterns; duplicate-spans warning; *"This feature is for advanced users only"*
> - Enable the OpenTelemetry Span Sensor for OneAgent (DT docs) — *"Existing tracers are replaced and will no longer work after you enable OpenTelemetry Java."*; *"Find and enable OpenTelemetry (Python) [Opt-In]"*
> - Technology support matrix (DT docs) — Go library tables; *"application binary must be dynamically linked unless you're using Go static monitoring"*
> - OpenTelemetry on AWS Lambda (DT docs)
> - OneAgent SDK for Java (Dynatrace GitHub) — serverless-not-supported guidance
> - otel4s (Typelevel GitHub), zio-telemetry (ZIO GitHub)
> - OpenTelemetry Host Monitoring extension (DT docs) — *"This extension automatically generates entities for hosts and their running processes."*

14. Deploying to Windows Servers

Short answer: the framework applies to Windows Server unchanged. The OneAgent-and-OTel decision is driven by *runtime* (Java / .NET / Node / Python / Go / PHP / Ruby) and *deployment model* (long-lived host vs serverless vs async-fragmenting), **not by the operating system**. A Java service gets the same answer on Windows as on Linux. What changes on Windows is the *weighting* of a few factors and the *deployment mechanics* — covered below.



OneAgent is a first-class citizen on Windows Server

OneAgent installs from an **MSI package and runs as a Windows `SYSTEM` service** — the install creates the registry entries that start it at boot. Server Core installations are supported on Long-Term Servicing Channel releases when OneAgent is installed in headless mode

(or monitored in an app-only scenario). It delivers the same full-stack deep monitoring it does on Linux: host metrics, process detection, runtime metrics, log streams, Smartscape topology, and PurePath — none of which require a code change. For the current supported Windows Server version list (and the Server Core / LTSC footnote), check the Technology support page linked in the Sources block rather than hard-coding a version here — the matrix moves with each release train.

The decisive Windows factor — .NET Framework and IIS

This is the one place where "which OS" materially changes the recommendation, because the classic, Windows-only **.NET Framework** stack lives here.

- **OneAgent auto-instruments classic .NET Framework** (4.5.2–4.8 and 3.5 SP1 supported; 4.5–4.5.1 limited) **and modern .NET** (.NET 5–10 and .NET Core 2.1–3.1), including **ASP.NET Framework, ASP.NET Core, WCF, .NET Remoting, gRPC, and the MSMQ client**. IIS application pools have built-in instrumentation rules — OneAgent injects into the `w3wp.exe` worker processes automatically.
- **OTel's .NET auto-instrumentation is strongest for ASP.NET Core** and is narrower, more manual, and version-sensitive for classic ASP.NET Framework. On IIS it requires per-application-pool environment-variable wiring.

The practical consequence: **on an estate that is predominantly legacy .NET Framework + IIS, OneAgent is the decisive low-effort path to deep coverage** — it instruments the classic stack that OTel covers least well, with no code change and no per-app-pool wiring. OpenTelemetry remains the right layer for *custom business spans* and for *open-standard instrumentation*; layer it alongside OneAgent (per §8), do not rip it out. For modern ASP.NET Core workloads the coverage gap closes and the choice reverts to the general framework in §9.

Three 1.343 additions that close .NET gaps this section leans on

The argument above — that OneAgent is the low-effort path on a .NET Framework / IIS estate — depends on how completely OneAgent covers the .NET publishing and hosting shapes actually in use. **OneAgent 1.343 (released 07/28/2026)** closes three of them. As always, **verify the agent version on the hosts concerned**; a fleet on 1.342 or earlier does not have these yet.

- **.NET single-file self-contained apps are now monitored.** This is the load-bearing one. Single-file self-contained publish (`PublishSingleFile` with a bundled runtime) was a known instrumentation gap, which mattered precisely because it is a common way to ship a self-hosted .NET service onto a Windows box without installing a runtime. A team that had hit that gap would reasonably have concluded OneAgent did not cover their deployment style and reached for OTel — the decision logic in this section would have sent them the wrong way. From 1.343 it is covered.
- **gRPC status codes for .NET** are now captured, so a failing gRPC call carries the status rather than surfacing as an opaque error. Relevant to the WCF-replacement pattern, where gRPC is the usual modern successor on Windows service stacks.
- **Kong Gateway Enterprise 3.10–3.14** is supported by the OneAgent NGINX module from OneAgent 1.343, with runtime instrumentation enabled (`DT_NGINX_FORCE_RUNTIME_INSTRUMENTATION=1` when starting Kong), which matters where a Windows-hosted .NET estate sits behind Kong rather than IIS ARR or a hardware load balancer.

None of this changes the section's conclusion — it strengthens it, by removing the exceptions that used to argue against it.

Logs and traces on Windows

Both tools deliver **logs and traces** on Windows — the difference is configuration effort and correlation fidelity.

Logs

- **OneAgent** auto-detects and ingests the **Windows Event Log** channels (System / Application / Security) and **text log files** (IIS logs, custom application logs). It is enabled by the *[Built-in] Windows system, application, and security logs* log-ingest rule, or included automatically under *[Built-in] Ingest all logs*. Each log line is auto-correlated to the host / process / service entity that produced it.
- **OpenTelemetry-only** reads the Event Log via the Collector's `windowseventlogreceiver` (the `channel:` field is required — e.g. `Application`, `System`, `Security`) and text logs via the `filelog` receiver, then exports OTLP. You configure each channel and path and own the attribute hygiene that ties logs back to entities.

Traces

- **OneAgent** PurePath auto-captures traces for .NET (including IIS `w3wp.exe`), Java, and Node on Windows with no code change. Critically, it traces the classic Windows service stack — **ASP.NET Framework, WCF, .NET Remoting, gRPC, and the MSMQ client** — which OTel's .NET auto-instrumentation covers least well.
- **OpenTelemetry-only** spans come from the .NET auto-instrumentation injector / SDK and export via OTLP; they correlate with OneAgent spans through W3C `traceparent` (per §8). Coverage of classic .NET Framework / WCF / MSMQ is narrower, so some

Windows service hops can be missing unless manually instrumented.

Smartscape dependency mapping on Windows

This is **where OneAgent adds the most on Windows**, and the reason to run OneAgent for topology even when OpenTelemetry owns the spans.

Dependency-mapping aspect	OneAgent on Windows	OpenTelemetry-only on Windows
Topology graph	Automatic Smartscape — host → process group → service, real-time	Service nodes and call edges from instrumented spans; host and process entities via the OpenTelemetry Host Monitoring extension (reference setup targets Linux — verify on Windows)
Service-to-service dependencies	Detected automatically for every instrumented service	Inferable only from span parent/child of *instrumented* calls
Process / host entity model	Automatic — every Windows process and host is an entity	Only with the OpenTelemetry Host Monitoring extension — verify on Windows
Windows service-stack hops (ASP.NET Framework, WCF, .NET Remoting, MSMQ)	Captured because OneAgent auto-instruments these (§6) → they appear in the map	Appear only if explicitly instrumented; classic Framework / WCF / MSMQ coverage is weak
Uninstrumented / third-party processes	Still placed on the map via host / process observation	No service node without spans; process entities only with the Host Monitoring extension
Configuration effort	Zero	Per-service instrumentation + attribute hygiene

Practical impact for a Windows estate: IIS + WCF + MSMQ + .NET Remoting is exactly the interaction style OpenTelemetry instruments least completely. An OTel-only deployment shows the hops you instrumented and a blank where you didn't — host and process topology needs the Host Monitoring extension on top of the Collector. OneAgent fills that gap with a complete, zero-config dependency map. If Smartscape dependency mapping matters to you (impact analysis, Davis RCA, blast-radius), **run OneAgent for topology and let OpenTelemetry own custom spans** — the §13 "layer, don't convert" recommendation applied to the dependency-map question.

Deployment mechanics on Windows

Concern	OneAgent	OpenTelemetry
Install unit	MSI; runs as a `SYSTEM` Windows service (`--unpack-msi` extracts the package + batch installer for scripted rollout)	Collector MSI registers a Windows service named "OpenTelemetry Collector" and an Event Log source; per-app .NET injector for in-process instrumentation
IIS-hosted apps	Auto-detected; instruments the application pools' `w3wp.exe` processes — no per-pool config	.NET auto-instrumentation requires environment variables set on each IIS application pool
App redeploy	Not required — agent attaches at the next process restart	SDK changes need a redeploy; auto-instrumentation changes need a process restart
Host / runtime metrics	Automatic	Add a `hostmetrics` receiver to a Collector running on the node
Coexistence	Use the §8 Span Sensor or OTLP pattern to avoid duplicate spans	Same — pick one ingestion path per process

Windows containers

Dynatrace documents no OneAgent path for Windows container workloads on Kubernetes. The Azure table of the technology-support matrix marks the AKS Operator rollout *"Windows Pods and Nodes unsupported"*, and every container auto-injection row in the matrix requires glibc or musl-libc. No Dynatrace page found (checked 10/02/2026) documents host OneAgent on a Windows Kubernetes node as a way to monitor its containers. Plan Windows container workloads as OTel-instrumented — the OTel SDK or .NET auto-instrumentation inside the container, exporting OTLP — or record them as an explicit support gap, and confirm with Dynatrace for your distribution. Plain Windows Server hosts outside Kubernetes are unaffected: the rest of this section applies to them.

Each conclusion below is a synthesis, not a documented recommendation: the .NET Framework / IIS call rests on the documented OneAgent/OTel coverage asymmetry, the Windows-container call on the matrix's AKS footnote and Linux-only container injection rows,

and the Smartscape rows on OneAgent's documented WCF / .NET Remoting / MSMQ instrumentation (§ 6) together with § 4's position that OneAgent fills in infrastructure topology automatically, while OTel data gets service topology from its spans.

> **Sources:**
> - Install OneAgent on Windows (DT docs) — *"Creates entries in the Windows Registry that start OneAgent as a `SYSTEM` service"*; `--unpack-msi` extraction for scripted install
> - .NET technology support (DT docs) — Framework 4.5.2–4.8 + 3.5 SP1 and modern .NET 5–10 + .NET Core 2.1–3.1 supported; IIS application-pools have built-in instrumentation rules
> - End-of-support news (DT docs) — listed under *2026-12-31 End of Support: ".NET Core 3.1 Vendor end of support: 2022-12-13 Last supported OneAgent version: 1.353"*; .NET 5, 6 and 7 are listed under the same date with the same last OneAgent version.
> - OneAgent platform and capability support matrix (DT docs)
> - Technology support (DT docs) — current supported Windows Server versions; LTSC footnote: *"Support includes Server Core installation (requires OneAgent installed in headless-mode) or monitored in app-only scenario."*; AKS footnote: *"Windows Pods and Nodes unsupported."*; .NET Remoting, WCF and MSMQ Client listed as *"All versions supported"*; container auto-injection rows: *"glibc or musl-libc required"*
> - OneAgent 1.343 release notes (DT docs) — released 07/28/2026; .NET single-file self-contained app monitoring, gRPC status codes for .NET, and *"The OneAgent NGINX module now supports Kong Gateway Enterprise versions 3.10–3.14."*
> - Install the Collector on Windows (opentelemetry.io) — MSI installs the Collector as a Windows service ("OpenTelemetry Collector") with an Event Log source
> - Windows event logs (DT docs) — OneAgent auto-detects System / Application / Security channels; enabled via the *[Built-in] Windows system, application, and security logs* rule or *[Built-in] Ingest all logs*
> - Windows Event Log Receiver (OpenTelemetry collector-contrib GitHub) — *"tails and parses logs from windows event log API"*; required `channel` field
> - OpenTelemetry Host Monitoring extension (DT docs) — *"This extension automatically generates entities for hosts and their running processes."*
> - Distributed traces — concepts (DT docs) — OneAgent auto-collects topology data and entity relationships for Smartscape; PurePath
> - **Derived:** every conclusion in this section combines a documented OneAgent/OTel coverage asymmetry with this entry's low-code-change thesis (§§ 4-6, 9); no page states them as recommendations

15. Runtime Applicability Map

Runtime-by-runtime — applicability of this guidance:

Runtime	Applies as written?	Notes
Java (classical threads, `CompletableFuture`, executors)	Yes	Reference case — OneAgent auto-instrumentation is strong
Java 21+ (Project Loom virtual threads)	Mostly	The OTel Java agent supports virtual threads; OneAgent documents virtual-thread support for CPU profiling — verify trace propagation on your agent version
Kotlin	Mostly	Coroutines have a fiber-like context flow; OTel's `kotlinx.coroutines` instrumentation handles it. OneAgent supports Kotlin Coroutines 1.10.2–2.1 (support matrix); verify other versions
Scala (classical — Akka, Play, plain `Future` / `ForkJoinPool`)	Yes	OneAgent auto-instrumentation is strong here
Scala (effect systems — ZIO, Cats Effect, fs2, http4s on IO)	Modified	See §7 — OTel via *otel4s* / *zio-telemetry* is technically required, not preferred
Groovy	Yes	Tracks Java; auto-instrumented by both
Clojure	Mostly	Same JVM auto-instrumentation; `core.async` channels have context issues comparable to coroutines
.NET Framework (4.5+ / classical ASP.NET)	Yes	OneAgent auto-instruments; OTel coverage is narrower for ASP.NET Framework. On IIS / Windows Server this is the decisive factor — see §14

Runtime	Applies as written?	Notes
.NET 6 / 7 / 8+ (modern .NET)	Yes	Both have strong coverage; `AsyncLocal<T>` flows trace context natively across `async` / `await`. .NET runtime-async needs OneAgent 1.345+ with Advanced .NET Instrumentation Mode
Node.js (JavaScript)	Yes	Both use `async_hooks` / `AsyncLocalStorage`
Node.js (TypeScript)	Yes	Same as JavaScript at runtime
Python (sync — Django, Flask)	Yes	Both have mature coverage
Python (asyncio — FastAPI, Starlette, Tornado)	Yes	Both use `contextvars`
Python (greenlet / gevent / eventlet monkey-patched)	Modified	Mixed coverage in both; verify per-library; OTel often the more reliable path
Go	Yes, for the libraries in OneAgent's Go matrix	OneAgent auto-injects 64-bit Go executables (1.323+) — `net/http`, SQL drivers, gRPC, Kafka, MongoDB, Redis, Cassandra and AWS SDK clients; the Span Sensor captures OTel Go spans; the Go OneAgent SDK has no custom tracing, so custom spans and other libraries come from OTel
PHP	Yes	OneAgent's PHP module is mature; OTel is newer but sufficient
Ruby	OTel-only at app level	Dynatrace's documented recommendation; OneAgent supplies host/process
Mobile native (iOS / Android)	Out of scope	Use Dynatrace Mobile Agent or OTel mobile SDKs — covered in the MOBL series
Frontend Web (browser JS)	Out of scope	Use Dynatrace RUM JavaScript or OTel browser SDK — covered in the WEBRUM series

Adjacent OTel ecosystem libraries to know about:

- **Java / Scala** — `opentelemetry-java-instrumentation` (Java agent), `otel4s` (Cats Effect-native), `zio-telemetry` (`zio-opentelemetry` module)
- **.NET** — `OpenTelemetry.Instrumentation.AspNetCore`, `OpenTelemetry.Instrumentation.Http`, `OpenTelemetry.Instrumentation.SqlClient`, `OpenTelemetry.Instrumentation.GrpcNetClient`
- **Node.js** — `@opentelemetry/auto-instrumentations-node`, `@opentelemetry/sdk-trace-node`, `@opentelemetry/context-async-hooks`
- **Python** — `opentelemetry-distro`, `opentelemetry-instrumentation-django`, `opentelemetry-instrumentation-fastapi`, `opentelemetry-instrumentation-aiohttp-client`
- **Go** — `go.opentelemetry.io/otel`, `otelhttp`, `otelgin`, `otelgrpc`, `otelpgx`, `otelmongo`
- **PHP** — `open-telemetry/opentelemetry-auto-laravel`, `open-telemetry/opentelemetry-auto-symfony`
- **Ruby** — `opentelemetry-instrumentation-all`, `opentelemetry-instrumentation-rails`, `opentelemetry-instrumentation-sinatra`
- **Cross-runtime** — `opentelemetry-collector` (vendor-neutral telemetry pipeline; common deployment pattern is app → Collector → Dynatrace OTLP)

Adjacent Dynatrace capabilities that depend on OneAgent:

- **Automatic infrastructure topology** — hosts, processes and containers in Smartscape without running a Collector (OTel data gets service topology on its own)
- **PurePath** — code-level distributed trace with method-level granularity on demand
- **Application Security** — Runtime Vulnerability Analytics and Runtime Application Protection, both on OneAgent deep monitoring
- **Live Debugger** — Java and Node.js code modules, OneAgent 1.309+
- **Automatic RUM injection** — OneAgent in full-stack mode inserts the RUM JavaScript into web pages
- **Real-User Monitoring ↔ backend** — linking HTML-document and resource requests to backend traces (Server-Timing header); XHR/Fetch and mobile requests link to OTel-captured traces as well
- **Process / host / log correlation** — automatic by virtue of agent placement

Davis AI is deliberately not on this list: it runs on OTel data too. OneAgent fills in the infrastructure topology it correlates against automatically.

Adjacent topic series in this notebook collection:

- **OTEL series** — OpenTelemetry integration deep dives
- **AIOPS series** — Davis AI, predictive / causal / generative intelligence
- **APPSEC series** — Application Security: Runtime Vulnerability Analytics and Runtime Application Protection
- **K8S series** — Dynatrace Operator, DynaKube, Kubernetes observability
- **MOBL series** — native mobile monitoring (iOS / Android)
- **WEBRUM series** — browser RUM and Core Web Vitals
- **DASH series** — dashboards over mixed OneAgent + OTel data
- **ADOPT series** — instrumentation maturity and adoption roadmap

> **Sources:**
>
> - Technology support (DT docs) — framework lists: Akka, Play, *"Kotlin Coroutines 1.10.2 - 2.1"*, the Go library tables
> - Frontend-backend linking (DT docs) — *"It works for distributed traces captured by OneAgent as well as those captured by OpenTelemetry."*, *"For web frontends with an OTel-instrumented backend, only XHR and Fetch requests can be linked to backend traces."* and *"Requests for the HTML document and its included resources can't be linked, because setting the Server-Timing header is not part of the OpenTelemetry specification."*
> - Java technology support (DT docs) — JVM runtime versions; virtual threads mentioned for CPU profiling only
> - OneAgent 1.345 release notes (DT docs) — .NET runtime-async: *"This requires the Advanced .NET Instrumentation Mode OneAgent feature to be enabled."*
> - Go OpenTelemetry walkthrough (DT docs) — the OpenTelemetry path for Go custom spans
> - Go technology support (DT docs) — *"Automatic injection and instrumentation of 64-bit Go executables on x86"*
> - OneAgent SDK for Java (Dynatrace GitHub) — *"The OneAgent SDK is not supported on serverless code modules, including those for AWS Lambda."* (the SDK, not OneAgent)
> - OpenTelemetry on AWS Lambda (DT docs) — Lambda extension covers Python/Node/Java
> - AWS Lambda integration (DT docs) — the Dynatrace Lambda layer's per-runtime support
> - otel4s (Typelevel GitHub), zio-telemetry (ZIO GitHub) — basis for *"Modified"* applicability rating on Scala effect systems
> - Runtime Vulnerability Analytics (DT docs) — *"Dynatrace OneAgent monitors loaded libraries, runtime components, and data flows in real time."*
> - Runtime Application Protection (DT docs) — *"Code-level visibility provided by OneAgent"*
> - Live Debugger (DT docs) — *"OneAgent version 1.309+ with Java Live-Debugger and Node.js Live-Debugger"*
> - RUM JavaScript injection (DT docs) — *"Once OneAgent is deployed in full-stack monitoring mode on a host, it by default automatically injects the RUM JavaScript"*
>
> The Clojure `core.async` note is **community guidance**, not vendor-documented.
>
> **A signal worth tracking for Rust (Dynatrace API 1.346, released 08/25/2026).** The API changelog adds **`RUST`** **and** **`TOKIO`** as `serviceTechnology` values on the Request Attributes endpoints and as technology values on `GET /extensions/{technology}/availableHosts` (Early Access).
>
> Read this carefully, because it is a **configuration-surface** change, not an announcement of OneAgent auto-instrumentation for Rust. What it establishes is that Dynatrace now models Rust and Tokio as first-class technology values — which is a prerequisite for deeper support, and useful today if you are naming or attributing Rust services. For the decision this entry frames, **Rust remains an OpenTelemetry-instrumented runtime**: it is absent from the auto-instrumentation coverage in §6 and the async-context-propagation matrix in §7, and nothing in 1.346 changes that. Treat this as a reason to re-check the supported-technologies page over the next few sprints, not as a reason to revisit an OTel decision for Rust today.
>
> **OneAgent 1.347 (published 10/01/2026, staged rollout from 09/30/2026 — verify the deployed agent version) adds Rust and Tokio process technology detection** — OneAgent will recognise Rust and Tokio processes. That is detection, not tracing: the OTel conclusion for Rust application spans stands.
>
> **Sources:** Dynatrace API changelog version 1.346 (DT docs) — `RUST` and `TOKIO` added to the serviceTechnology and extension-technology enums, read 08/28/2026. OneAgent 1.347 release notes (DT docs) — *"Implemented Rust and Tokio process technology detection."* (*"Rollout start on Sep 30, 2026"*, read 10/02/2026)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against the current Dynatrace documentation.